LLM 基础系列 · 第 0 册

# 神经网络前置知识详解

从感知机到注意力：一条可复用的知识主线

MLP · CNN · RNN · LSTM · GRU · 词嵌入 · Seq2Seq · 注意力

重写版 · 2026 年 10 月

先建立地图，再深入山谷

# 重写说明

这一版不是对原稿的逐段润色，而是重新组织叙述。

原稿有一条很好的主线：从神经元、词嵌入、CNN、RNN/LSTM，走到 Seq2Seq 与注意力。问题在于，它更像一条为 Transformer 服务的“垂直补给线”：数学准备不足，历史与分类被压缩，若干手算例子对不上，工程细节又集中堆在最后。读者容易记住零散公式，却说不清每个零件为什么在那里。

重写版做了四个决定。

**第一，先给地图，再给山谷。** 第 1 章回答“神经网络是什么、有哪些、彼此什么关系”；第 2 章补齐读懂后面公式所需的数学；第 3 章才进入第一个手算。这样读者始终知道自己在整张地图的什么位置。

**第二，保留五个必须亲手走通的推导。** 两层 MLP 的反向传播、卷积的手算与反传、RNN 的 BPTT、LSTM 的门控、Bahdanau 注意力。这五处不省略数字，因为“看懂”和“算过”是两回事。

**第三，把广度放在该放的地方。** 分词、位置编码、GRU、归一化、生成模型、自监督、图网络、状态空间模型、对齐与评估，不展开成百科，但都给出“它解决什么问题、代价是什么”的位置。读者以后遇到新名词，知道它属于哪一类。

**第四，把“事实”与“观点”分开。** 年份、论文归属、实验数字尽量回查原始论文；有争议或依赖配置的结论，用“通常”“取决于”“在本例中”限定。原稿中的主要事实与计算错误，集中列在独立的《审校报告》里，不塞进正文。

> **阅读约定**
> 向量用粗体小写字母表示，矩阵用粗体大写字母表示；$\odot$ 是逐元素乘法；上标 $(l)$ 表示第 $l$ 层，下标 $t$ 表示序列第 $t$ 步。带“可选”的段落可以跳过，不影响主线。

# 这本书想让你带走什么

不是背完所有公式，而是形成四种能力：

- 看到一个模型，能说出它注入的**归纳偏置**：局部性、顺序性、平移等变、内容路由，还是别的什么；
- 看到一个训练问题，能分清是**优化**问题、**泛化**问题，还是**工程**问题；
- 看到一个手算例子，能检查**形状、数值、梯度**是否自洽；
- 看到一个新技术，能问：它替代了旧方案的哪条假设，又付出了什么代价。

# 三条阅读路线

| 路线 | 读什么 | 目标 |
| --- | --- | --- |
| 第一遍：地图 | 第 1、2 章通读；第 3 – 7 章读“原理、过程、例子”；第 8、9 章浏览 | 建立完整框架，知道每个概念的位置 |
| 第二遍：推导 | 第 2.2 节、第 3.5 节、第 5.2 节、第 6.2 节、第 7.2 节 | 亲手走通反向传播、卷积、BPTT、注意力 |
| 第三遍：工程 | 第 3.7 节、第 8 章，配合代码与清单 | 能实现、训练、调试一个小网络，并理解大模型训练的系统约束 |

## 与后续分册的边界

本册只负责“前置知识”：基本积木、主流架构的动机与训练常识。Transformer 的完整结构、预训练目标、解码策略、分布式训练账本、RAG 与 Agent，留给后续分册。

读完本册，你应当能够：

- 用“数据—模型—损失—优化—泛化”描述任何监督学习系统；
- 手推一个小型 MLP 的前向与反向，并解释每个梯度的形状；
- 说清词嵌入、卷积、循环网络、注意力各自注入了什么先验；
- 解释 LSTM 为什么只能缓解、不能根治长依赖；
- 把 Bahdanau 注意力与 Transformer 的 Q/K/V 逐项对应；
- 看懂训练循环里的关键工程决定，并知道常见故障如何排查。

# 目录


## 第一部分　先看森林

第 1 章　神经网络是什么：三个视角、三次浪潮与一张地图　5

第 2 章　读懂后面公式所需的最小数学　10

## 第二部分　基本积木

第 3 章　从神经元到多层感知机　16

第 4 章　表征与词嵌入：符号如何变成向量　24

## 第三部分　结构与序列

第 5 章　卷积网络：把图像先验写进结构　31

第 6 章　循环网络、LSTM 与 GRU：给网络装上记忆　38

第 7 章　Seq2Seq 与注意力：Transformer 的前夜　44

## 第四部分　从模型到系统

第 8 章　训练工程：从一行梯度下降到分布式系统　50

第 9 章　地图的边缘：生成、自监督与新一代架构　55

## 附录

附录 A　公式与符号速查　58

附录 B　术语中英对照　61

附录 C　自检题与参考答案　63

附录 D　延伸阅读　65

# 第 1 章　神经网络是什么：三个视角、三次浪潮与一张地图

> 在学会一个神经元之前，先知道神经元在整台机器里的位置。

## 1.1 三个视角，缺一不可

“神经网络”这个词容易让人想到两幅画面：一幅是大脑里的神经元，另一幅是屏幕上密密麻麻的连线。两幅画都不够准确。更有用的理解方式，是同时从三个角度看同一个模型。

**它是函数。** 一个神经网络是一个带参数的函数

$$f_\theta : \mathcal{X} \to \mathcal{Y},$$

输入一张图片、一段文本或一行日志，输出一个类别、一个数值或下一个 token 的概率分布。训练就是在候选函数中找一个在数据上表现好的 $f_\theta$。这个视角回答“模型能表达什么”。

**它是表征。** 网络的每一层都把输入变换成新的表示：

$$h^{(1)} = \phi_1(x), \quad h^{(2)} = \phi_2(h^{(1)}), \quad \dots$$

最后一层在这些表示上做预测。词嵌入把离散 token 变成连续向量，CNN 的深层特征从边缘、纹理走向部件和物体，Transformer 的上下文表示让同一个词在不同句子里有不同向量。这个视角回答“模型学到了什么”。

**它是计算图。** 前向传播是一串张量运算；只要每个运算可微，就可以用链式法则把损失对参数的梯度反向传播。现代框架的核心抽象之一，就是记录这张图并自动求导。这个视角回答“模型怎么训练”。

> **三个视角合起来**
>
> 函数告诉你表达力，表征告诉你学到了什么，计算图告诉你如何优化。读一篇新论文时，分别问这三个问题，通常比逐行抄公式更快抓住本质。

## 1.2 所有监督学习共享同一个循环

不管模型是 MLP、CNN 还是 Transformer，训练循环的骨架都一样：

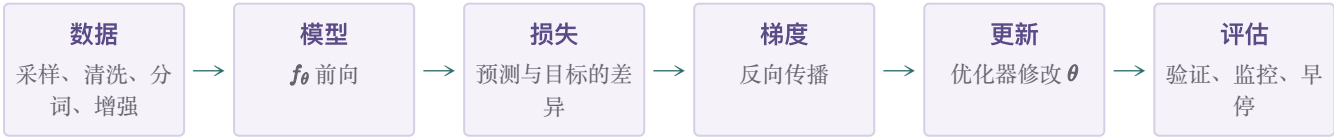



图 1–1　训练循环。真实系统还会包含日志、检查点、分布式通信与故障恢复。

形式化地，给定训练集 $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$，模型输出 $\hat{y}_i = f_\theta(x_i)$，损失为 $\ell(\hat{y}_i, y_i)$。经验风险最小化试图求解

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} \ell\big(f_\theta(\boldsymbol{x}_i), \boldsymbol{y}_i\big).$$

实际训练用一小批样本估计梯度：

$$\boldsymbol{g}_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla_\theta \ell_i(\theta_t), \qquad \theta_{t+1} = \theta_t - \eta_t \boldsymbol{g}_t.$$

后面会看到动量、自适应学习率、权重衰减等变体，但“用一小批数据估计梯度，再沿负梯度走一步”是所有变体的骨架。

> **训练损失低，不等于模型好**
>
> 训练只是把已见数据的损失压低；模型真正要做的是**泛化**。过拟合、数据泄漏、分布偏移都会让训练指标漂亮而真实效果很差。优化与泛化是两个问题，第 2.4 节会分开讲。

## 1.3 三次浪潮：技术突破从来不是“算法一出现就赢”

神经网络的历史常被压缩成“1943 神经元 → 1958 感知机 → 1986 反向传播 → 2012 AlexNet → 2017 Transformer”。这条线大体不错，但它掩盖了三次重要的反复。

| 年份 | 事件 | 为什么重要 |
| --- | --- | --- |
| 1943 | McCulloch 与 Pitts 提出形式神经元 | 把神经活动写成逻辑运算 |
| 1958 | Rosenblatt 提出感知机 | 第一个能通过数据调整权重的线性分类器 |
| 1969 | Minsky 与 Papert 出版《Perceptrons》 | 严格分析单层感知机的局限，常被误读为“多层也不行” |
| 1974 | Werbos 的博士论文使用反向传播 | 反向传播的思想早于 1986 年 |
| 1986 | Rumelhart、Hinton、Williams 推广反向传播 | 多层网络训练变得可行 |
| 1989 | Cybenko、Hornik 等证明万能逼近定理 | 一个隐藏层、足够宽，可以逼近连续函数 |
| 1989 – 1998 | LeCun 等发展卷积网络，LeNet-5 用于手写数字 | 卷积、权值共享、梯度训练落地 |
| 1997 | Hochreiter 与 Schmidhuber 提出 LSTM | 门控与细胞状态缓解梯度消失 |
| 2000 | Gers 等加入遗忘门 | 今天常说的“标准 LSTM”包含这一补充 |
| 2006 | Hinton 等用逐层预训练重启深层网络 | 在 ReLU、GPU 和大数据普及前的一条路径 |
| 2012 | AlexNet 赢得 ImageNet | 大数据 + GPU + ReLU + Dropout 的组合引爆深度学习 |
| 2014 | Seq2Seq、GRU、Bahdanau 注意力、GAN、VAE | 序列、生成与注意力集中突破 |
| 2015 | ResNet、BatchNorm、Luong 注意力 | 极深网络可训练，注意力继续成熟 |
| 2017 | Transformer | 用自注意力替代循环，开启大规模并行预训练 |
| 2018 | BERT、GPT、ELMo | 预训练 + 微调成为 NLP 主流 |

| 年份 | 事件 | 为什么重要 |
| --- | --- | --- |
| 2020 | GPT-3、ViT、缩放定律 | 大模型、视觉 Transformer、规模化规律 |
| 2022 | ChatGPT、Chinchilla | 对齐与“数据—参数匹配”成为关键词 |

表 1–1　关键节点。年份按论文公开或广泛传播的时间标注，不同文献可能有一年差异。

三次浪潮的起落，各有各的原因。

**第一次浪潮（1950s – 1960s）** 由感知机和早期学习规则点燃。单层感知机可以学习线性可分问题，但当人们发现它连 XOR 都表示不了，热情迅速降温。这里有一个常见误解：Minsky 与 Papert 证明的是**单层**感知机的局限，不是多层网络不可能；真正缺的是训练多层网络的有效方法。

**第二次浪潮（1980s – 1990s）** 由反向传播点燃。多层网络可以训练了，CNN 在手写数字上落地，LSTM 在序列任务上出现。但深层网络仍然难训：激活饱和、没有归一化、数据规模小、算力不足。SVM 等核方法在中小规模数据上更稳，连接主义再次退潮。

**第三次浪潮（2012 年以后）** 不是某一个算法的胜利，而是数据、算力、算法与工程的合流。ImageNet 提供了百万级标注；GPU 提供了并行算力；ReLU、Dropout、更好的初始化让深层网络可训练；开源框架降低了实现门槛。此后，Transformer 与大模型把“规模”推到了新的量级。

> **历史不是一条直线**
>
> AlexNet 不是第一个 CNN，Transformer 也不是第一个注意力模型。一个想法能否胜出，往往取决于数据规模、算力、工程实现、社区生态和时机。学历史的意义不是背年份，而是理解：**技术突破通常是“旧想法 + 新条件”**。

## 1.4 一张分类表：同一个模型，可以从很多角度看

“神经网络”是一个家族，不是单个模型。下面这张表给出最常用的分类维度。初学时不必记住所有名词，但要建立“同一件事可以从多个角度描述”的意识。

| 维度 | 类别 | 典型代表 | 核心问题 |
| --- | --- | --- | --- |
| 学习信号 | 监督学习 | 分类、回归 | 有标签，学习 $\boldsymbol{x} \to \boldsymbol{y}$ |
|  | 无监督学习 | 聚类、降维、密度估计 | 没有标签，寻找结构 |
|  | 自监督学习 | 掩码建模、对比学习、下一 token 预测 | 从数据本身构造监督信号 |
|  | 强化学习 | 策略梯度、Q-learning | 通过奖励学习序列决策 |
| 输出 | 判别式 | 逻辑回归、分类 CNN | 建模 $\boldsymbol{p(y \mid x)}$ |
|  | 生成式 | VAE、GAN、扩散、自回归模型 | 建模 $\boldsymbol{p(x)}$，能采样新数据 |
| 结构 | 前馈 | MLP | 信息单向流动，无环 |
|  | 卷积 | CNN | 局部连接、权值共享、平移等变 |
|  | 循环 | RNN、LSTM、GRU | 隐状态沿时间传递 |

| 维度 | 类别 | 典型代表 | 核心问题 |
| --- | --- | --- | --- |
| | 注意力 | Transformer | 任意位置按内容动态交互 |
| | 图 | GCN、GAT | 处理非欧几里得结构 |
| | 状态空间 | S4、Mamba | 用线性动力系统建模长序列 |
| 训练方式 | 从零训练 | 小模型、特定任务 | 数据与算力需求可控 |
| | 预训练 + 微调 | BERT、GPT、ViT | 先学通用表示，再适配任务 |
| | 指令微调 / 对齐 | InstructGPT、DPO | 让模型遵循指令、符合偏好 |

表 1–2　常见分类维度。一个模型可以同时属于多个类别，例如 GPT 是“生成式、自回归、注意力、预训练 + 对齐”的模型。

“深度”通常指从输入到输出经过的可学习变换层数，而不是参数量或计算量。深度带来的不是简单的“表达力线性增加”，而是**组合性**：深层网络可以复用中间特征，用层次化的方式表示复杂函数。但深度也带来优化困难，于是有了残差连接、归一化和更好的初始化。

## 1.5 依赖地图：一条主链，三条支线

如果把本册的知识画成一张图，它不是线性列表，而是一棵有主干的树。

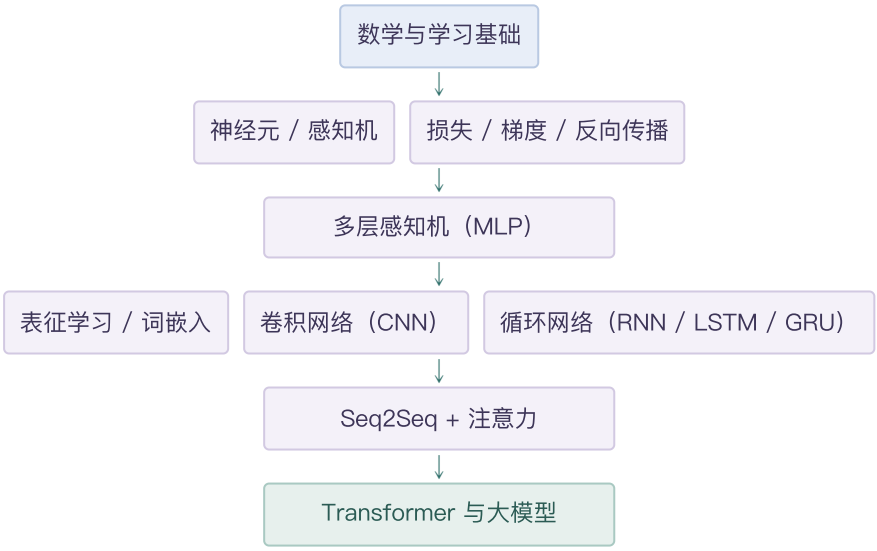



图 1–2　本册的依赖地图。CNN 是视觉支线，不是 Transformer 主干的必经阶段；但它的权值共享、层级特征与残差连接深刻影响了后续架构。

这条主链的每一环，都是被上一环的“痛点”逼出来的：

- 单层感知机只能画一条直线，连 XOR 都分不开；把多个神经元叠成 **MLP**，并加入非线性激活，才有了逼近非线性函数的能力。
- MLP 只接受数值向量，而文字是离散符号；**词嵌入**把 token 映射为可学习的连续向量。
- MLP 要求定长、无序的输入；语言和音频是变长序列；**RNN** 用隐状态把历史带进下一步。
- RNN 的固定隐状态形成信息瓶颈；**Seq2Seq + 注意力**让解码器按需回看编码器的所有状态。
- 注意力的并行性与长距离建模能力，最终催生了 **Transformer**。

### 为什么还要学“过时”的技术

一个常见的疑问是：RNN、LSTM 在大模型里已经被 Transformer 取代，为什么还要学？至少有四个理由。

**第一，不理解旧范式的痛点，就不理解新范式的设计。** “RNN 串行、无法并行”只有一行字，但只有亲手推过 $h_t$ 对 $h_{t-1}$ 的依赖、算过 50 步梯度连乘，才知道这句话的分量。

**第二，注意力不是凭空出现的。** Bahdanau 注意力的“打分 → softmax → 加权和”与 Transformer 的 Q/K/V 注意力是同一数学结构的两种写法。学过前身，Q/K/V 就是“老朋友换了打分函数”。

**第三，工程遗产仍在服役。** 梯度裁剪、初始化、门控、残差、嵌入查表、序列打包，这些实践从 RNN 时代沉淀下来，今天仍在用。

**第四，很多场景仍然适合旧架构。** 流式语音、边缘设备、低功耗传感器、短序列时间序列，RNN/CNN/状态空间模型可能比 Transformer 更合适。架构没有绝对优劣，只有与数据、算力、延迟约束的匹配。

> **不要把 CNN 当成 Transformer 的“前一站”**
>
> CNN 不在“感知机 → RNN → 注意力”的主链上，它是处理网格结构数据的另一条支线。把它硬塞进线性进化史，会掩盖一个关键事实：**架构的分化来自数据结构的差异，而不是谁比谁更“高级”。**

## 1.6 本册的边界

本册覆盖神经网络的基本概念、历史与分类；读懂后续公式所需的数学；MLP、CNN、RNN/LSTM/GRU、Seq2Seq 与注意力；训练系统的关键工程问题；以及一张通往生成模型、自监督、图网络、状态空间模型与对齐技术的扩展地图。

本册不替代严格的机器学习理论教材，也不替代完整的 Transformer 与 LLM 工程手册。第一遍读不要死磕所有推导：先读“原理、过程、例子”，把图表和例子跑通；第二遍再回到公式，亲手推一遍；第三遍带着工程问题读第 8 章。

> **把这一章放回地图**
>
> 神经网络不是“很多神经元连起来”这么简单。它是**函数、表征与计算图**的三位一体；不同架构的差异，本质上是注入了不同的归纳偏置。记住这一点，后面的 MLP、CNN、RNN、注意力就不会变成四个孤立的名词。

# 第 2 章　读懂后面公式所需的最小数学

> 这一章不追求数学系的严谨，只补齐“看到公式不慌”所需的最小工具箱。

很多神经网络教程假设读者“线代记得、概率没忘、微积分还在”。现实是，大多数工程师的数学已经还给老师了。更麻烦的是，神经网络用到的数学不是按教材顺序出现的：它同时用到矩阵乘法、链式法则、概率分布和数值稳定性。本章按“用得到”的顺序组织。

## 2.1 向量、矩阵与形状

一个 $d$ 维向量

$$\boldsymbol{x} = [x_1, \dots, x_d]^\top \in \mathbb{R}^d$$

可以看作空间中的一个点，也可以看作一个特征表示。神经网络里，一张图片、一个 token、一个隐藏状态，最终都是向量。

两个向量的**点积**是

$$\boldsymbol{x}^\top \boldsymbol{y} = \sum_{i=1}^{d} x_i y_i = \|\boldsymbol{x}\|\,\|\boldsymbol{y}\| \cos\theta.$$

点积衡量方向相似度：同向为正，正交为 0，反向为负。**余弦相似度**去掉长度，只保留方向：

$$\cos(\boldsymbol{x}, \boldsymbol{y}) = \frac{\boldsymbol{x}^\top \boldsymbol{y}}{\|\boldsymbol{x}\|_2 \|\boldsymbol{y}\|_2}.$$

> **为什么注意力用点积打分**
>
> 注意力要判断“查询”和“键”有多相关。把两者映射成向量后，点积是最便宜的相关性度量：一次矩阵乘就能算出所有位置对的分数。它不一定是“最正确”的度量，但在高维空间中配合学习到的投影矩阵，效果很好，而且极适合 GPU。

矩阵是线性变换。一个 $m \times n$ 矩阵 $\boldsymbol{A}$ 把 $\mathbb{R}^n$ 映射到 $\mathbb{R}^m$：

$$\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}, \qquad y_i = \sum_{j=1}^{n} A_{ij} x_j.$$

矩阵乘法 $\boldsymbol{C} = \boldsymbol{A}\boldsymbol{B}$ 的元素是

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj},$$

形状规则是 $(m \times n)(n \times p) \to (m \times p)$。神经网络里最常见的形状约定是：一个 batch 有 $B$ 个样本、每个样本 $d$ 维，堆成 $\boldsymbol{X} \in \mathbb{R}^{B \times d}$；一层线性变换写成

$$\boldsymbol{Z} = \boldsymbol{X}\boldsymbol{W} + \boldsymbol{b}, \qquad \boldsymbol{W} \in \mathbb{R}^{d \times h}.$$

一次矩阵乘处理整个 batch，而不是写 $\boldsymbol{B}$ 次循环。这就是“向量化”的本质，也是 GPU 能发挥作用的前提。

| 符号 | 常见形状 | 含义 |
| --- | --- | --- |
| $\boldsymbol{X}$ | $\boldsymbol{B} \times \boldsymbol{d}$ | 一个 batch 的输入 |
| $\boldsymbol{W}$ | $\boldsymbol{d} \times \boldsymbol{h}$ | 线性层权重 |
| $\boldsymbol{b}$ | $\boldsymbol{h}$ | 偏置，广播到每一行 |
| $\boldsymbol{Z}$ | $\boldsymbol{B} \times \boldsymbol{h}$ | 线性层输出 |
| $\boldsymbol{Y}$ | $\boldsymbol{B} \times \boldsymbol{c}$ | 输出 |

表 2–1　最常见的 batch 形状。读代码时先看形状，往往比看变量名更快定位 bug。

后面还会用到几个概念：

- **范数**：衡量向量大小。L2 范数 $\|\boldsymbol{x}\|_2 = \sqrt{\sum_i x_i^2}$ 最常用；L1 倾向稀疏；L∞ 关注最大分量。
- **秩与奇异值**：描述矩阵保留了多少独立方向。低秩意味着压缩；SVD 把矩阵分解为“旋转—缩放—旋转”。
- **特征值与谱半径**：若 $\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v}$，$\lambda$ 是特征值；谱半径 $\rho(\boldsymbol{A}) = \max_i |\lambda_i|$ 决定循环网络梯度连乘是衰减还是爆炸（第 6.2 节）。

## 2.2 导数、链式法则与计算图

一元函数 $f(x)$ 的导数描述“输入动一点，输出动多少”。多元函数对第 $i$ 个变量的偏导排成向量，就是**梯度**：

$$\nabla_{\boldsymbol{x}} f = \left[ \frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_d} \right]^{\top}.$$

梯度指向函数上升最快的方向，因此沿负梯度走，函数值下降最快。

若 $y = g(x)$、$z = f(y)$，链式法则是

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}.$$

多元情形写成矩阵：

$$\nabla_{\boldsymbol{x}} z = \boldsymbol{J}_g^{\top} \nabla_{\boldsymbol{y}} z, \qquad (\boldsymbol{J}_g)_{ji} = \frac{\partial y_j}{\partial x_i}.$$

反向传播就是“从标量损失出发，按链式法则逐层左乘雅可比转置”。

### 一个最小例子

设 $z = (2x + 1)^2$。前向拆成

$$u = 2x + 1, \qquad z = u^2.$$

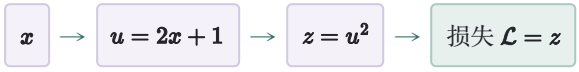
反向从 $\partial z / \partial z = 1$ 开始：

$$\frac{\partial z}{\partial u} = 2u, \qquad \frac{\partial u}{\partial x} = 2,$$

所以

$$\frac{\partial z}{\partial x} = 2u \cdot 2 = 4(2x+1).$$

当 $x=1$ 时，$u=3$，梯度为 12。如果直接展开 $z = 4x^2 + 4x + 1$，求导得 $8x + 4$，代入也是 12。计算图做法的价值在于：无论组合多复杂，都可以按同样规则自动扩展。




反向：$\partial \mathcal{L}/\partial z = 1 \to \partial \mathcal{L}/\partial u = 2u \to \partial \mathcal{L}/\partial x = 2u \cdot 2$

图 2–1　前向记录中间值，反向按链式法则逐层相乘。这就是自动微分的骨架。

### 梯度、雅可比与海森矩阵

- 梯度 $\nabla f$：标量函数对向量输入的偏导，形状与输入相同。
- 雅可比 $\boldsymbol{J}$：向量函数对向量输入的偏导矩阵，形状为“输出维度 × 输入维度”。
- 海森矩阵 $\boldsymbol{H}$：标量函数的二阶偏导，$H_{ij} = \partial^2 f / \partial x_i \partial x_j$，描述曲率。

二阶泰勒展开为

$$f(\boldsymbol{x} + \Delta \boldsymbol{x}) \approx f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})^{\top} \Delta \boldsymbol{x} + \frac{1}{2} \Delta \boldsymbol{x}^{\top} \boldsymbol{H} \Delta \boldsymbol{x}.$$

如果梯度为 0 且 $\boldsymbol{H}$ 正定，该点是局部极小；如果 $\boldsymbol{H}$ 既有正特征值又有负特征值，则是鞍点。深度网络的损失曲面通常高度非凸，鞍点比局部极小更常见。

## 2.3 概率、交叉熵与 softmax

神经网络处理的数据通常被看作随机变量的样本。分类任务最常用的分布是**类别分布**：$p(y = k) = p_k$，$\sum_k p_k = 1$。

**最大似然估计**选择让数据最可能出现的参数：

$$\theta_{\text{MLE}} = \arg\max_{\theta} \sum_{i=1}^{N} \log p(x_i \mid \theta).$$

取对数是因为连乘容易下溢，而且对数把乘积变成求和。**在监督学习中，最小化负对数似然等价于最小化交叉熵。**

信息熵衡量不确定性：

$$H(p) = -\sum_{x} p(x) \log p(x).$$

交叉熵衡量“用分布 $q$ 编码来自 $p$ 的数据”所需的平均信息量：

$$H(p, q) = -\sum_{x} p(x) \log q(x).$$

KL 散度衡量两个分布的差异：

$$D_{\mathrm{KL}}(p\|q)=\sum_x p(x)\log\frac{p(x)}{q(x)}=H(p,q)-H(p).$$

因为 $H(p)$ 与模型参数无关，最小化交叉熵等价于最小化 KL。分类任务中，真实标签通常是 one-hot，交叉熵退化为

$$\mathcal{L}_{\mathrm{CE}}=-\log q(y_{\text{true}}),$$

也就是“正确类别的预测概率越高越好”。

softmax 把 $K$ 个分数变成概率：

$$\mathrm{softmax}(\boldsymbol{z})_i=\frac{\exp(z_i)}{\sum_{j=1}^{K}\exp(z_j)}.$$

它非负、和为 1，对全体加同一个常数不变。温度参数 $\tau$ 可以调节平滑程度：

$$\mathrm{softmax}(\boldsymbol{z}/\tau)_i=\frac{\exp(z_i/\tau)}{\sum_j\exp(z_j/\tau)}.$$

$\tau\to 0$ 趋近 argmax，$\tau\to\infty$ 趋近均匀分布。

> **softmax 的输出不一定是“校准过的概率”**
>
> softmax 保证输出是一个合法的概率分布，但不保证它等于真实后验概率。一个模型可能给出 0.99 的置信度，实际准确率只有 0.7。这叫**校准**问题，需要温度缩放、可靠性图等方法单独评估。

### 数值稳定：log-sum-exp

直接算 $\log\sum_i\exp(z_i)$，$z_i$ 很大会溢出，很小会下溢。标准技巧是减最大值：

$$\log\sum_{i=1}^{K}\exp(z_i)=m+\log\sum_{i=1}^{K}\exp(z_i-m),\qquad m=\max_i z_i.$$

因为 $\exp(z_i-m)\le 1$，求和不会溢出；至少有一项为 1，也不会全部下溢。softmax 的实现同样先减最大值。这个技巧在交叉熵、注意力、语言模型里无处不在。

## 2.4 优化与泛化：两件不同的事

训练的目标通常写成

$$\min_\theta\ \mathcal{L}(\theta)=\frac{1}{N}\sum_{i=1}^{N}\ell\big(f_\theta(\boldsymbol{x}_i),\boldsymbol{y}_i\big)+\lambda\Omega(\theta),$$

第一项让模型拟合数据，第二项 $\Omega$ 是正则项（如 L2 权重衰减）。

**随机梯度下降**用一小批估计梯度：

$$g_t = \frac{1}{B}\sum_{i\in\mathcal{B}_t}\nabla_\theta \ell_i(\theta_t), \qquad \theta_{t+1} = \theta_t - \eta_t g_t.$$

动量累积历史梯度，减少震荡：

$$v_{t+1} = \beta v_t + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_{t+1}.$$

Adam 为每个参数维护一阶矩和二阶矩：

$$m_{t+1} = \beta_1 m_t + (1-\beta_1) g_t,$$

$$v_{t+1} = \beta_2 v_t + (1-\beta_2) g_t^2,$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_{t+1}}{\sqrt{\hat{v}_{t+1}} + \epsilon}.$$

其中 $\hat{m}$、$\hat{v}$ 是偏差修正后的矩估计。AdamW 把权重衰减从梯度更新中解耦，通常比“在损失里加 L2”更适合大模型。

优化解决“怎么把训练损失降下来”，泛化解决“未见数据上表现如何”。两者不是一回事。

- **欠拟合**：模型太简单或训练不足，训练误差和验证误差都高。
- **过拟合**：模型记住训练噪声，训练误差低但验证误差高。
- **偏差—方差**：简单模型偏差大、方差小；复杂模型偏差小、方差大。
- **数据泄漏**：训练时用到了预测时不可能获得的信息，例如用未来数据预测过去、在全集上拟合标准化参数。
- **分布偏移**：测试分布与训练分布不同，可能表现为协变量偏移、标签偏移或概念偏移。

> **双下降**
>
> 现代深度网络常常“参数比样本还多”，按经典偏差—方差理论应该严重过拟合，但测试误差在插值点附近可能再次下降。这被称为**双下降**。它提醒我们：经典理论是重要直觉，但不是全部。

## 2.5 浮点数与数值稳定

| 格式 | 总位数 | 指数位 | 尾数位（显式） | 典型用途 |
|---|---|---|---|---|
| FP32 | 32 | 8 | 23 | 传统训练与推理 |
| FP16 | 16 | 5 | 10 | 混合精度训练、推理 |
| BF16 | 16 | 8 | 7 | 大模型训练，范围接近 FP32 |
| FP8 | 8 | 5 或 4 | 2 或 3 | 新一代加速器，需精细缩放 |

表 2–2　常见浮点格式。尾数位决定精度，指数位决定动态范围。

FP16 的最大值约为 **65504**，训练中梯度稍大就可能溢出；BF16 的最大值约为 $\mathbf{3.4\times10^{38}}$，范围接近 FP32，但精度更低。混合精度训练通常用 FP16/BF16 做矩阵乘，用 FP32 保存主权重和累加，并配合损失缩放防止梯度下

溢。

数值稳定的常见技巧：

- softmax / log-sum-exp 先减最大值；
- 交叉熵用框架的 `log_softmax` + `nll_loss`，不要先 softmax 再 log；
- 归一化分母加小常数 $\epsilon$；
- 梯度范数超过阈值时按比例裁剪；
- 初始化控制方差，避免激活和梯度指数级放大/缩小。

> **把这一章放回地图**
>
> 点积衡量相似度，矩阵乘法是线性变换与批量运算，链式法则是反向传播的骨架，交叉熵是分类的默认损失，softmax 把分数变成分布，数值稳定决定这些公式能不能在 GPU 上跑。后面的每一个模型，都是这些积木的不同组合。

# 第 3 章　从神经元到多层感知机

> 这一章只做一件事：让你亲手训练一个两层网络一步，并且每个数字都能对上。

## 3.1 一个神经元能做什么

生物神经元大致可以抽象为：树突接收信号，胞体整合，超过阈值后沿轴突发放。1943 年，McCulloch 和 Pitts 把它写成

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right).$$

其中 $x_i$ 是输入，$w_i$ 是权重，$b$ 是偏置，$f$ 是激活函数。权重表示“这个输入有多重要”，偏置表示“激活的难易程度”。注意符号：如果写成 $\sum_i w_i x_i - \theta$，$\theta$ 是阈值；写成 $+b$ 时，$b = -\theta$。因此 $b$ 越大，神经元越容易被激活，而不是“门槛越高”。

> **生物类比要适可而止**
>
> 人工神经元借用了“加权求和 + 非线性”的直觉，但它不是生物神经元的忠实模型：没有时间动力学、没有突触可塑性、没有复杂的树突计算。把神经网络说成“模拟大脑”是过度简化；更准确的说法是“受生物启发的一类可微函数近似器”。

1958 年，Rosenblatt 提出感知机。它使用阶跃函数：

$$\hat{y} = \mathrm{step}(\boldsymbol{w}^{\top}\boldsymbol{x} + b), \qquad \mathrm{step}(z) = \begin{cases} 1, & z \ge 0, \\ 0, & z < 0. \end{cases}$$

感知机是一个线性分类器：用超平面 $\boldsymbol{w}^{\top}\boldsymbol{x} + b = 0$ 把空间分成两半。二维时是一条直线，三维时是一个平面，高维时是一个超平面。

对于误分类样本，感知机按

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta y_i \boldsymbol{x}_i, \qquad b \leftarrow b + \eta y_i$$

更新，其中 $y_i \in \{-1, +1\}$。如果数据线性可分，感知机在有限步内找到一个分离超平面；如果线性不可分，它不会收敛。

### XOR：一条直线分不开的四点

XOR 的真值表是：

| $x_1$ | $x_2$ | $y$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |

| | | |
|---|---|---|
| 1 | 1 | 0 |

四个点呈对角分布，不存在一条直线把两类分开。形式化地，若存在 $w_1, w_2, b$ 满足

$$
\begin{aligned}
0 \cdot w_1 + 0 \cdot w_2 + b &< 0, \\
1 \cdot w_1 + 1 \cdot w_2 + b &< 0, \\
0 \cdot w_1 + 1 \cdot w_2 + b &\geq 0, \\
1 \cdot w_1 + 0 \cdot w_2 + b &\geq 0,
\end{aligned}
$$

则前两式给出 $b < 0$ 且 $w_1 + w_2 < -b$，后两式给出 $w_1, w_2 \geq -b$，相加得 $w_1 + w_2 \geq -2b$。与 $w_1 + w_2 < -b$ 结合，推出 $b > 0$，矛盾。因此单层感知机无法表示 XOR。

> **历史误读**
>
> Minsky 与 Papert 在 1969 年严格分析的是单层感知机的局限，并没有证明多层网络无法解决 XOR。多层网络可以；当时缺的是训练方法和算力。把“AI 寒冬”完全归因于 XOR 是过度简化，但 XOR 确实是“为什么需要隐藏层”的经典例子。

## 3.2 隐藏层：把空间“折”一下

如果一条直线不够，就用多条直线组合。考虑两个隐藏神经元：

$$
h_1 = \text{step}(x_1 + x_2 - 0.5), \qquad h_2 = \text{step}(x_1 + x_2 - 1.5).
$$

$h_1$ 在至少一个输入为 1 时激活，相当于 OR；$h_2$ 在两个输入都为 1 时激活，相当于 AND。再用输出神经元

$$
\hat{y} = \text{step}(h_1 - h_2 - 0.5),
$$

就得到 XOR：

| $x_1$ | $x_2$ | $h_1$ (OR) | $h_2$ (AND) | $h_1 - h_2$ | $\hat{y}$ |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 0 | 0 |

这个例子说明：隐藏层不是“多了一层线性变换”，而是把输入映射到一个新的表示空间。在原始空间里 XOR 线性不可分；在 $(h_1, h_2)$ 空间里，四个点变成 $(0,0), (1,0), (0,1), (1,1)$，再用一条直线就能分开。深度网络的力量，很大程度上来自这种“逐层改变表示，让原本难分的问题变得容易分”。

## 3.3 激活函数：非线性从哪里来

如果没有激活函数，多层线性变换仍然是线性变换：

$$\boldsymbol{h} = \boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1, \qquad \hat{\boldsymbol{y}} = \boldsymbol{W}_2\boldsymbol{h} + \boldsymbol{b}_2 = \underbrace{\boldsymbol{W}_2\boldsymbol{W}_1}_{\boldsymbol{W}}\boldsymbol{x} + \underbrace{\boldsymbol{W}_2\boldsymbol{b}_1 + \boldsymbol{b}_2}_{\boldsymbol{b}}.$$

无论叠多少层，最终都等价于一个线性变换。激活函数的作用，是在层与层之间插入非线性。

Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}, \qquad \sigma'(x) = \sigma(x)(1-\sigma(x)).$$

值域 $(0,1)$，输出可以解释为二分类概率。导数最大值为 $0.25$（$x=0$ 处），$|x|$ 很大时导数趋近 $0$，容易梯度消失；输出不是零中心。

Tanh

$$\tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}, \qquad \tanh'(x) = 1 - \tanh^2(x).$$

值域 $(-1,1)$，零中心，导数最大值为 $1$，但两端仍饱和。比 sigmoid 更适合隐藏层。

ReLU

$$\mathrm{ReLU}(x) = \max(0,x), \qquad \mathrm{ReLU}'(x) = \begin{cases} 1, & x>0, \\ 0, & x<0. \end{cases}$$

正半轴不饱和，计算只需一次比较，是深度学习工业化后的事实标准。缺点是负半轴梯度为 $0$，可能出现“死亡 ReLU”。

Leaky ReLU / PReLU

$$\mathrm{LeakyReLU}(x) = \begin{cases} x, & x>0, \\ \alpha x, & x\le 0, \end{cases}$$

其中 $\alpha$ 是小常数（如 0.01），或作为可学习参数（PReLU）。负半轴保留一个小斜率，缓解死亡 ReLU。

GELU

$$\mathrm{GELU}(x) = x\Phi(x),$$

其中 $\Phi$ 是标准正态分布的累积分布函数。常用近似为

$$\mathrm{GELU}(x) \approx 0.5x\left(1+\tanh\left(\sqrt{\frac{2}{\pi}}\left(x+0.044715x^3\right)\right)\right).$$

GELU 在 Transformer 的 FFN 中很常见，形状比 ReLU 平滑。

SiLU / Swish

$$\mathrm{SiLU}(x) = x\sigma(x),$$

平滑、非单调，在部分现代架构中使用。

Softmax 通常用于多分类输出层，不是隐藏层的常规激活：

$$\mathrm{softmax}(\boldsymbol{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

| 激活函数 | 值域 | 导数特点 | 优点 | 缺点 |
|---|---|---|---|---|
| Sigmoid | $(0,1)$ | 最大 0.25，两端饱和 | 概率解释直观 | 梯度消失、非零中心 |
| Tanh | $(-1,1)$ | 最大 1，两端饱和 | 零中心 | 仍会饱和 |
| ReLU | $[0,\infty)$ | 正区 1，负区 0 | 计算快、不饱和 | 死亡 ReLU |
| Leaky ReLU | $(-\infty,\infty)$ | 负区小斜率 | 缓解死亡 ReLU | 多一个超参数 |
| GELU | 约 $(-0.17,\infty)$ | 平滑 | Transformer 常用 | 计算稍贵 |
| SiLU | 约 $(-0.28,\infty)$ | 平滑、非单调 | 部分模型表现好 | 计算稍贵 |
| Softmax | $(0,1)$，和为 1 | 耦合所有分量 | 输出概率分布 | 只适合输出层 |

表 3–1　常见激活函数。选择时要看任务、深度、数值精度和训练稳定性，而不是只看“哪个最新”。

> **SwiGLU 不是激活函数**
>
> SwiGLU 是一种门控前馈网络结构，不是像 ReLU 那样逐元素作用的激活函数。它通常写成 $\mathbf{FFN}(x) = (\mathbf{SiLU}(xW_1) \odot xW_3)W_2$，包含三个线性变换。把 SwiGLU 叫作“激活函数”会掩盖它的门控结构，也会算错参数量。

## 3.4 前向传播与损失

一个多层感知机由若干全连接层堆叠而成。第 $l$ 层：

$$\boldsymbol{z}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)}, \qquad \boldsymbol{a}^{(l)} = f^{(l)}\big(\boldsymbol{z}^{(l)}\big),$$

其中 $\boldsymbol{a}^{(0)} = \boldsymbol{x}$，最后一层输出 $\hat{\boldsymbol{y}} = \boldsymbol{a}^{(L)}$。如果第 $l$ 层有 $n_{l-1}$ 个输入、$n_l$ 个输出，参数量为

$$n_{l-1}n_l + n_l.$$

例如输入 784，隐藏层 256，输出 10：

$$(784 \times 256 + 256) + (256 \times 10 + 10) = 200{,}960 + 2{,}570 = 203{,}530.$$

损失函数把“预测”和“目标”的差异变成一个标量。回归常用均方误差：

$$\mathcal{L}_{\mathrm{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2.$$

二分类常用二元交叉熵，配合 sigmoid：

$$\mathcal{L}_{\mathrm{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\big[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\big].$$

多分类常用交叉熵，配合 softmax：

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N}\sum_{i=1}^{N} \log \text{softmax}(\boldsymbol{z}_i)_{y_i}.$$

实现时不要先算 softmax 再取 log，直接用框架的 `cross_entropy`（内部结合 log-softmax），否则容易数值下溢。

> **损失与输出激活要匹配**
>
> 回归用 MSE/MAE，二分类用 sigmoid + BCE，多分类用 softmax + CE。把 softmax 和 MSE 混用不是绝对不行，但梯度性质更差、解释也不自然。框架的 `BCEWithLogitsLoss`、`CrossEntropyLoss` 把激活和损失合并，既稳定又高效。

## 3.5 反向传播：完整手算

### 一般公式

对第 $l$ 层，定义损失对预激活的梯度

$$\boldsymbol{\delta}^{(l)} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}^{(l)}}.$$

则

$$\boldsymbol{\delta}^{(L)} = \nabla_{\boldsymbol{a}^{(L)}}\mathcal{L} \odot \boldsymbol{f}^{(L)\prime}(\boldsymbol{z}^{(L)}),$$

$$\boldsymbol{\delta}^{(l)} = \left(\boldsymbol{W}^{(l+1)\top}\boldsymbol{\delta}^{(l+1)}\right) \odot \boldsymbol{f}^{(l)\prime}(\boldsymbol{z}^{(l)}),$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{(l)}} = \boldsymbol{\delta}^{(l)}\boldsymbol{a}^{(l-1)\top}, \qquad \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{(l)}} = \boldsymbol{\delta}^{(l)}.$$

矩阵形式下，$\boldsymbol{\delta}^{(l)}$ 与 $\boldsymbol{z}^{(l)}$ 同形；$\partial\mathcal{L}/\partial\boldsymbol{W}^{(l)}$ 与 $\boldsymbol{W}^{(l)}$ 同形。梯度形状必须与参数形状一致，这是排查反向传播 bug 的第一条规则。

### 一个两层网络的一步训练

为了把所有数字算清楚，我们使用一个最小的两层网络：

$$z_1 = w_1 x + b_1, \qquad h = \sigma(z_1), \qquad \hat{y} = w_2 h + b_2.$$

损失采用

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2.$$

取 $x = 1$、$y = 1$、$w_1 = 1$、$b_1 = 0$、$w_2 = 2$、$b_2 = 0$。

前向：

$$z_1 = 1, \qquad h = \sigma(1) \approx 0.7311,$$

$$\hat{y} = 2 \cdot 0.7311 \approx 1.4621, \qquad \mathcal{L} = \frac{1}{2}(0.4621)^2 \approx 0.1068.$$

反向：

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y = 0.4621.$$

输出层：

$$\frac{\partial \mathcal{L}}{\partial w_2} = 0.4621 \times 0.7311 \approx 0.3378, \qquad \frac{\partial \mathcal{L}}{\partial b_2} \approx 0.4621.$$

回传：

$$\frac{\partial \mathcal{L}}{\partial h} = 0.4621 \times 2 \approx 0.9242,$$

$$\frac{\partial \mathcal{L}}{\partial z_1} = 0.9242 \times 0.7311 \times 0.2689 \approx 0.1817.$$

隐藏层参数：

$$\frac{\partial \mathcal{L}}{\partial w_1} \approx 0.1817, \qquad \frac{\partial \mathcal{L}}{\partial b_1} \approx 0.1817.$$

四个梯度汇总：

| 参数 | 梯度 | 数值 |
|---|---|---|
| $w_2$ | $\partial \mathcal{L}/\partial w_2$ | 0.3378 |
| $b_2$ | $\partial \mathcal{L}/\partial b_2$ | 0.4621 |
| $w_1$ | $\partial \mathcal{L}/\partial w_1$ | 0.1817 |
| $b_1$ | $\partial \mathcal{L}/\partial b_1$ | 0.1817 |

取学习率 $\eta = 0.5$，四个参数都更新：

$$w_2' = 1.8311, \quad b_2' = -0.2311, \quad w_1' = 0.9091, \quad b_1' = -0.0909.$$

用新参数再前向：

$$z_1' = 0.8183, \quad h' = \sigma(0.8183) \approx 0.6939,$$

$$\hat{y}' = 1.8311 \times 0.6939 - 0.2311 \approx 1.0395,$$

$$\mathcal{L}' = \frac{1}{2}(1.0395 - 1)^2 \approx 0.00078.$$

损失从 $0.1068$ 降到 $0.00078$，下降约 $99.3\%$。这一次，四个参数都参与了更新，前向与反向的每个数字都能对上。

> **原稿这里错在哪**
>
> 原稿计算了 $b_1$ 的梯度 $0.36$，却在更新时只改了 $w_1$ 和 $w_2$，然后用 $z_1' = w_1' x$ 重新前向，相当于把 $b_1$ 的更新丢掉了；同时 $b_2$ 的梯度也没有用于更新。因此“损失从 0.212 降到 0.026、下降 88%”的结论不成立。本版改用 $\frac{1}{2}$ 均方误差，并完整更新四个参数。

### 梯度检查

反向传播写完后，用有限差分做数值检查：

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta_i + \epsilon) - \mathcal{L}(\theta_i - \epsilon)}{2\epsilon}.$$

中心差分误差是 $O(\epsilon^2)$。取 $\epsilon \approx 10^{-5}$，比较解析梯度与数值梯度的相对误差；大于 $10^{-4}$ 通常说明链式法则或形状写错了。Dropout、BatchNorm 等训练时随机的层，做检查要切到评估模式或固定随机种子。

## 3.6 万能逼近与初始化

**万能逼近定理**的一个常见版本是：

> 设 $\varphi$ 是一个非常数、有界、单调递增的连续函数。对于紧集 $K \subset \mathbb{R}^n$ 上的任意连续函数 $f$ 和任意 $\epsilon > 0$，存在一个单隐藏层网络
>
> $$g(\boldsymbol{x}) = \sum_{j=1}^{N} v_j \varphi(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j),$$
>
> 使得 $\sup_{\boldsymbol{x} \in K} |f(\boldsymbol{x}) - g(\boldsymbol{x})| < \epsilon$。

Cybenko（1989）证明了 sigmoid 型激活的版本，Hornik 等（1989）推广到更一般的激活函数。定理说明：一个隐藏层、足够多的神经元，理论上可以逼近紧集上的任意连续函数。

但要注意：**存在性，不是可学习性**；宽度可能非常大；只保证在紧集上；不保证泛化；激活函数要满足条件。万能逼近是“表达力”的下界，不是“训练效率”的上界。

### 初始化：打破对称性，控制方差

如果同一层所有参数初始化为相同值，同一输入下所有隐藏神经元输出相同，梯度也相同，更新后仍然相同。网络退化成一个很窄的模型。初始化必须打破对称性。

设输入 $x_i$ 独立、均值 0、方差 $\sigma_x^2$，权重 $w_i$ 独立、均值 0、方差 $\sigma_w^2$，线性输出 $z = \sum_i w_i x_i$ 的方差为

$$\mathrm{Var}(z) = \sum_i \mathrm{Var}(w_i x_i) = n \sigma_w^2 \sigma_x^2.$$

为了让信号逐层传播时方差大致不变，需要 $n\sigma_w^2 = 1$，即 $\sigma_w^2 = 1/n$。这是简化版 Xavier。完整的 Xavier 还考虑反向传播，取输入和输出的折中：

$$\sigma_w^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}.$$

ReLU 会把一半输出置零，方差大约减半。为了补偿，把权重方差放大一倍：

$$\sigma_w^2 = \frac{2}{n_{\text{in}}}.$$

这就是 **He 初始化**。

| 初始化 | 权重方差 | 适用激活 | 直觉 |
| --- | --- | --- | --- |
| Xavier/Glorot | $2/(n_{\text{in}}+n_{\text{out}})$ | tanh、sigmoid | 前向、反向方差都尽量不变 |
| He/Kaiming | $2/n_{\text{in}}$ | ReLU、Leaky ReLU | 补偿 ReLU 置零一半的方差损失 |
| 标准正态 | $1$ | 无特定 | 往往导致深层网络方差爆炸或衰减 |

表 3–2　常见初始化。关键是方差量级，而不是具体采样方式。

在极深的 Transformer 中，残差分支输出常被乘以一个缩放因子，例如 $1/\sqrt{N}$ 或 $1/\sqrt{2N}$，其中 $N$ 是残差层数。具体用哪个取决于实现约定，不是普遍定律。

## 3.7 MLP 没有消失：它住进了 Transformer

Transformer 的每个块里都有一个前馈网络（FFN）。经典形式是

$$\mathrm{FFN}(\boldsymbol{x}) = \boldsymbol{W}_2\ \mathrm{GELU}(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2,$$

这就是一个两层 MLP：先升维，再降维。它以逐位置的方式作用：对序列中每个 token 的向量独立处理，不混合不同 token 的信息。混合 token 信息的任务由注意力完成。

以 $d = 4096$、$d_{\text{ff}} = 11008$ 为例：

- 标准两层 FFN：$2 \times 4096 \times 11008 \approx 90.2$M 参数；
- SwiGLU 门控 FFN：$3 \times 4096 \times 11008 \approx 135.3$M 参数；
- 一个注意力层（Q/K/V/O 四个 $d \times d$ 矩阵）：$4 \times 4096^2 \approx 67.1$M 参数。

因此，标准 FFN 约占“注意力 + FFN”参数的 **57%**，SwiGLU 版本约占 **67%**。常说“FFN 占 Transformer 约三分之二参数”，更准确的说法是：在采用 SwiGLU 等门控结构、且忽略嵌入层时，FFN 接近三分之二；具体比例取决于模型配置。

> **嵌入层与投影矩阵也是可学习参数**
>
> 词嵌入矩阵 $\boldsymbol{E} \in \mathbb{R}^{V \times d}$、注意力的 Q/K/V/O 投影、FFN 的权重，全部由反向传播更新。第 3.5 节手算的流程，就是这些矩阵中每一个数字的来源。7B 模型、词表 32000、$d = 4096$ 时，嵌入矩阵约有 $32000 \times 4096 \approx 1.31$ 亿参数，bf16 下约 **0.26** GB。

> **把这一章放回地图**
>
> MLP 是后面所有架构的共同祖先：线性变换、非线性激活、损失、反向传播、初始化。CNN 把线性变换换成卷积，RNN 把层沿时间复用，Transformer 把 MLP 放进每个块里做逐位置变换。你在这里算过的每一步，后面都会以不同形状再次出现。

# 第 4 章　表征与词嵌入：符号如何变成向量

> 神经网络只吃向量。文字是离散符号。词嵌入就是两者之间的桥。

## 4.1 one-hot 的直觉与局限

假设词表有 $V$ 个词。最直接的编码是 one-hot：每个词用一个 $V$ 维向量表示，只有对应位置为 1，其余为 0。

$$\text{猫} = [1,0,0], \qquad \text{狗} = [0,1,0], \qquad \text{鱼} = [0,0,1].$$

one-hot 简单、无歧义，但有两个致命问题。

**维度高、稀疏。** 真实词表常有 3 万到 10 万甚至更多 token，每个词都是超稀疏向量，存储和计算都不划算。

**没有语义相似度。** 任意两个不同词的 one-hot 点积为 0，余弦相似度也为 0。“猫”和“狗”的距离，与“猫”和“微积分”的距离完全一样。向量空间里没有任何“意义”的结构。

我们需要一种编码，让语义相近的词，向量也相近。这就是词嵌入要解决的问题。

## 4.2 分布式假设：看邻居知词义

语言学中有一个朴素而深刻的观察，通常追溯到 Firth（1957）：

> You shall know a word by the company it keeps.（看一个词和谁常在一起，就知道它的意思。）

“猫”常和“喵”“宠物”“毛”共现；“狗”常和“汪”“宠物”“遛”共现。两者的上下文高度重叠，因此语义相近。反过来，一个不认识的词，看它十次出现的上下文，也能猜个八九不离十。

这就是**分布式假设**：词义由它的上下文分布决定。把这条假设变成优化目标，就可以从海量文本中学习词向量：为每个词学一个低维稠密向量，使得“上下文相似的词，向量也相似”。

> **“分布式”有两个容易混淆的含义**
>
> 在词嵌入语境中，“分布式表示”指一个概念由多个维度共同表示，而不是一个维度对应一个概念。它和“分布式系统”没有关系。one-hot 是“局部表示”，词向量是“分布式表示”。

## 4.3 word2vec：用预测任务学向量

2013 年，Mikolov 等提出 word2vec，把词向量学习变成预测任务。它有两个版本：

- CBOW：用上下文词预测中心词；
- Skip-gram：用中心词预测上下文词。

### Skip-gram 的目标

给定中心词 $c$ 和上下文词 $o$，Skip-gram 用 softmax 建模：

$$p(o \mid c) = \frac{\exp(\boldsymbol{u}_o^\top \boldsymbol{v}_c)}{\sum_{w \in V} \exp(\boldsymbol{u}_w^\top \boldsymbol{v}_c)},$$

其中 $\boldsymbol{v}_c$ 是词 $c$ 作为中心词时的向量，$\boldsymbol{u}_o$ 是词 $o$ 作为上下文词时的向量。每个词有两个向量，训练结束后通常取中心词向量，或把两套向量平均/拼接。

给定语料，Skip-gram 最大化所有“中心词—上下文词”对的概率：

$$\max \sum_{(c,o) \in \mathcal{D}} \log p(o \mid c).$$

点积越大，说明中心词与上下文词越匹配，softmax 概率越大。训练就是让真实共现对的点积变大，让随机词对的点积变小。

### 负采样：把多分类变成二分类

分母要对整个词表求和。词表 10 万时，每个样本都要算 10 万次指数，代价太高。**负采样**把“10 万选一”改造成一系列二分类：

- 正样本：真实共现对 $(c, o)$，标签为 1；
- 负样本：从噪声分布随机采 $K$ 个词 $n_k$，标签为 0。

目标函数变为

$$\mathcal{L} = -\log \sigma(\boldsymbol{u}_o^\top \boldsymbol{v}_c) - \sum_{k=1}^{K} \log \sigma(-\boldsymbol{u}_{n_k}^\top \boldsymbol{v}_c).$$

第一项让正样本的点积变大；第二项让负样本的点积变小。$K$ 通常取 5 – 20，噪声分布常取词频的 $3/4$ 次方，以降低高频词的采样概率。**层次 softmax** 是另一种近似：把词表组织成二叉树，把“选哪个词”变成沿树走左右分支的一系列二分类，复杂度从 $O(V)$ 降到 $O(\log V)$。

### 手算一步负采样

设二维玩具向量：

$$\boldsymbol{v}_c = (1, 0), \qquad \boldsymbol{u}_o = (1, 1), \qquad \boldsymbol{u}_n = (0, 1),$$

学习率 $\eta = 0.1$，只取一个负样本。

正样本打分 $s_o = 1$，$\sigma(1) \approx 0.7311$；负样本打分 $s_n = 0$，$\sigma(0) = 0.5$。损失为

$$\mathcal{L} = -\log \sigma(1) - \log \sigma(0) \approx 0.3133 + 0.6931 = 1.0064.$$

对 $\boldsymbol{v}_c$ 求梯度。正样本项：

$$\frac{\partial}{\partial \boldsymbol{v}_c}[-\log \sigma(s_o)] = \big(\sigma(s_o) - 1\big)\boldsymbol{u}_o \approx -0.2689\,(1, 1).$$

负样本项：

$$\frac{\partial}{\partial \boldsymbol{v}_c}[-\log \sigma(-s_n)] = \sigma(s_n)\boldsymbol{u}_n = 0.5\,(0, 1).$$

合起来：

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{v}_c} \approx (-0.2689, 0.2311).$$

更新：

$$\boldsymbol{v}_c' = \boldsymbol{v}_c - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{v}_c} \approx (1.0269, -0.0231).$$

验证：正样本点积从 1 变为 1.0269，变大了；负样本点积从 0 变为 −0.0231，变小了。海量语料上重复这个过程，语义就逐渐“凝聚”进向量的几何结构。

> **word2vec 不是“把词义分解成维度”**
>
> 词向量的每一维通常没有单独的可解释含义。语义分布在所有维度的组合中；你不能指着第 37 维说“这是动物性”。词向量的可解释性要靠类比、聚类、可视化等间接方法，而不是逐维命名。

## 4.4 GloVe、fastText 与上下文表示

### GloVe：全局共现 + 加权最小二乘

GloVe（2014）直接把全局共现统计作为目标。设 $\boldsymbol{X}_{ij}$ 是词 $\boldsymbol{j}$ 出现在词 $\boldsymbol{i}$ 上下文中的次数，GloVe 试图让

$$\boldsymbol{w}_i^\top \tilde{\boldsymbol{w}}_j + b_i + \tilde{b}_j \approx \log \boldsymbol{X}_{ij}.$$

损失函数为

$$J = \sum_{i,j=1}^{V} f(\boldsymbol{X}_{ij}) \left( \boldsymbol{w}_i^\top \tilde{\boldsymbol{w}}_j + b_i + \tilde{b}_j - \log \boldsymbol{X}_{ij} \right)^2,$$

其中 $f(\boldsymbol{X}_{ij})$ 用来降低极高频共现的权重（例如 $f(x) = \min(1, (x/x_{\max})^{3/4})$）。直觉是：共现概率的比值编码了语义关系，而点积可以拟合对数共现。

### fastText：把词拆成子词

fastText（2016/2017）用字符 n-gram 表示词：

$$\boldsymbol{v}_w = \sum_{g \in \mathcal{G}(w)} \boldsymbol{z}_g,$$

其中 $\mathcal{G}(w)$ 是词 $\boldsymbol{w}$ 的字符 n-gram 集合。这样即使一个词没在训练集中出现，只要它的子词见过，也能得到向量，缓解 OOV；对形态丰富的语言、拼写变体、罕见词更友好。

### 从静态词向量到上下文表示

word2vec、GloVe、fastText 学到的都是**静态词向量**：一个词不管出现在什么句子里，向量都一样。这显然不够：“苹果”在“吃苹果”和“苹果发布新手机”里意思不同，“bank”在“河岸”和“银行”里意思不同。

**上下文词向量**根据整句话动态计算每个 token 的表示：

- ELMo（2018）：用双向 LSTM 语言模型，取各层隐状态的组合；
- BERT（2018）：用 Transformer 编码器 + 掩码语言建模，得到深层双向表示；
- GPT 系列：用 Transformer 解码器 + 自回归语言建模，每个位置的表示只依赖左侧上下文。

今天的大模型嵌入，通常是 Transformer 某层（或某几层）的隐藏状态，而不是 word2vec。word2vec 的历史地位在于：它第一次大规模证明了“分布式词向量 + 简单预测任务”可以学到有语义几何的表示。

| 方法 | 表示 | 是否上下文相关 | 主要特点 |
| --- | --- | --- | --- |
| one-hot | 稀疏高维 | 否 | 简单，无语义相似度 |
| 共现 + SVD | 低维稠密 | 否 | 可解释，依赖全局统计 |
| word2vec | 低维稠密 | 否 | 预测式、效率高、负采样 |
| GloVe | 低维稠密 | 否 | 全局共现、加权最小二乘 |
| fastText | 低维稠密 | 否 | 子词，缓解 OOV |
| ELMo | 深层表示 | 是 | 双向 LSTM |
| BERT/GPT | 深层表示 | 是 | Transformer，预训练 |

表 4–1　词表示方法的演化。静态词向量仍适合某些轻量任务，但现代 NLP 的主流是上下文表示。

## 4.5 嵌入层：数学上是矩阵乘，工程上是查表

现代神经网络通常把嵌入层写成一个矩阵

$$\boldsymbol{E} \in \mathbb{R}^{V \times d},$$

其中 $\boldsymbol{V}$ 是词表大小，$\boldsymbol{d}$ 是嵌入维度。token $\boldsymbol{i}$ 的嵌入向量是 $\boldsymbol{E}$ 的第 $\boldsymbol{i}$ 行：

$$\boldsymbol{e_i} = \boldsymbol{E}[i, :].$$

数学上，它等价于 one-hot 向量乘矩阵：

$$\boldsymbol{e_i} = \boldsymbol{E}^{\top} \boldsymbol{q_i},$$

其中 $\boldsymbol{q_i}$ 是第 $\boldsymbol{i}$ 个 one-hot 列向量。但工程上绝不会真的构造 $\boldsymbol{V}$ 维 one-hot 再做矩阵乘——那会浪费大量乘加。实现就是一次数组索引：

```
embedding = E[token_id]   # 形状 (d,)
```

这是“数学上写成矩阵乘、工程上实现为查表”的经典案例。

### 反向传播：只有被查到的行收到梯度

设一个 batch 中 token $\boldsymbol{i}$ 的嵌入为 $\boldsymbol{e_i}$，损失对它的梯度为 $\boldsymbol{g_i}$。则

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{E}[j, :]} = \begin{cases} \boldsymbol{g_i}, & j = i, \\ 0, & j \neq i. \end{cases}$$

如果一个 batch 中同一个 token 出现多次，梯度会累加。实现上，这等价于 `scatter_add`：把每个位置的梯度加到对应行。**稀疏更新**是嵌入层的天然特征。

但要注意：如果嵌入矩阵和输出层**权重绑定**（weight tying），同一个矩阵既作为输入嵌入，又作为输出投影，那么输出侧的梯度会作用于所有行（至少是所有被预测到的类别），稀疏性就被打破。现代大模型有时绑定、有时不绑定，取决于设计。

### 参数量与显存

7B 模型、词表 $V = 32000$、$d = 4096$：

$$V \times d = 32000 \times 4096 = 131{,}072{,}000 \approx 1.31 \text{ 亿参数}.$$

bf16（每参数 2 字节）下约

$$131{,}072{,}000 \times 2\ \text{B} \approx 262\ \text{MB} \approx 0.26\ \text{GB}.$$

这只是嵌入矩阵。如果再加输出投影、优化器状态（Adam 的动量与二阶矩）、梯度，实际显存会更大。参数量估算时，不要把“参数量”与“显存”混为一谈：训练时每个参数往往对应多份状态。

## 4.6 分词与位置：token 不等于 word

在进入嵌入层之前，文本要先被切成 **token**。分词器决定词表、序列长度和模型看到的基本单位。

### 三种粒度

- **词级**：按空格/标点切词。词表大、OOV 多、形态变化难处理。
- **字符级**：按字符切。词表小、序列长、语义单元太细。
- **子词级**：介于两者之间，常用 BPE、WordPiece、SentencePiece、Unigram。

### BPE 的简化流程

1. 从字符（或字节）开始，每个字符是一个 token；
2. 统计相邻 token 对的出现频率；
3. 把频率最高的对合并成一个新 token；
4. 重复，直到达到预设的词表大小。

例如 `low`、`lower`、`lowest` 中，`l o w` 可能先合并成 `low`，再学出 `er`、`est` 等子词。GPT-2 使用 byte-level BPE，把文本先转成字节序列，因此可以表示任意 Unicode 字符，包括 emoji 和罕见符号。

### WordPiece 与 SentencePiece

- **WordPiece**：BERT 使用，合并准则基于似然提升，而不是单纯频率；未登录词用 `##` 表示续接子词。
- **SentencePiece**：把空格也当作符号，直接在原始文本上训练，不依赖预分词；支持 BPE 和 Unigram 两种算法，适合多语言。
- **Unigram**：从大词表开始，逐步删除对似然贡献小的 token，得到概率化的子词表；T5 使用。

### 分词带来的工程问题

序列长度按 token 计，不是按字符或词。中文、代码、数字的 token 效率不同；同一段文本，不同分词器会产生不同 token 数；数字和代码容易被切碎，影响算术、格式和标识符建模；某些语言的 token 效率低，同样内容需要更多 token，成本和延迟更高。把分词当成“随便切一下”，是很多跨语言和代码任务的隐性 bug 来源。

### 位置：注意力为什么需要位置信息

词嵌入只告诉模型“这个 token 是什么”，没有告诉它“这个 token 在哪里”。如果直接把一组词向量喂给注意力，打乱顺序，注意力输出（在忽略 mask 的情况下）可能不变——因为点积和加权和本身对位置没有偏好。语言里顺序至关重要：“狗咬人”和“人咬狗”词相同，意思相反。

因此，Transformer 需要**位置编码**。常见方案：

**正弦位置编码**

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right),$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right).$$

不同频率的正弦/余弦给每个位置一个独特模式，且相对位移可以通过线性变换表示。原版 Transformer 使用这种方案。

**可学习绝对位置嵌入**

$$\boldsymbol{x}_t = \boldsymbol{e}_t + \boldsymbol{p}_t.$$

优点是简单、灵活；缺点是最大长度在训练时固定，超出长度需要外推或插值。

**相对位置与旋转位置编码**

- **相对位置偏置**：在注意力分数上加一个与相对距离有关的偏置，而不是把位置加到输入上。
- **RoPE**：把 query/key 向量按位置旋转一个角度，使得点积自然依赖相对位置 $\boldsymbol{m} - \boldsymbol{n}$。现代大模型广泛使用。
- **ALiBi**：在注意力分数上加一个与距离成正比的线性惩罚，鼓励关注近处；对长度外推较友好。

| 方案 | 类型 | 优点 | 局限 |
| --- | --- | --- | --- |
| 正弦编码 | 绝对、固定 | 无需学习，有一定外推性 | 表达力有限，实际外推仍受训练长度限制 |
| 可学习绝对 | 绝对、可学习 | 简单，效果好 | 最大长度固定 |
| 相对偏置 | 相对 | 直接建模距离 | 实现较复杂 |
| RoPE | 相对（旋转） | 相对位置性质好，广泛使用 | 长上下文仍需调参 |
| ALiBi | 相对（线性偏置） | 外推友好 | 表达力依赖任务 |

表 4–2　常见位置编码。没有一种方案在所有长度和任务上都最优；长上下文能力还取决于训练数据与目标。

## 4.7 词向量的几何与局限

训练好的词向量常表现出有趣的几何规律。最著名的例子是：

$$\boldsymbol{v}_{\text{king}} - \boldsymbol{v}_{\text{man}} + \boldsymbol{v}_{\text{woman}} \approx \boldsymbol{v}_{\text{queen}}.$$

“king 减 man”提取出“王室且男性”的方向，加上“女性”的方向，落点接近 queen。类似的类比还有“国家—首都”“单数—复数”等。这说明语义关系可以被编码成向量空间中的方向。

但要注意：类比通常只是近似，不是精确等式；词向量的几何会受语料、训练目标、维度、频率影响；低频词、歧义词、文化特定词的类比更不稳定；词向量会继承语料中的性别、种族、职业等偏见，不能把“几何关系”直接当作“事实关系”。

> **“king − man + woman = queen”不是定律**
>
> 它是一个著名的经验现象，不是每个模型、每个词对都成立。把类比结果当作语义事实，容易过度解释。更严谨的做法是报告具体模型、数据集、评估方法和置信区间，而不是只展示一个漂亮例子。

> **把这一章放回地图**
>
> 词嵌入把离散符号变成连续向量；word2vec 用预测任务学静态向量；GloVe/fastText 从共现和子词改进；BERT/GPT 产生上下文表示。嵌入层是“可学习的查表”，分词器决定模型看见什么，位置编码让注意力知道顺序。下一章的 CNN、RNN，都是在这套表示之上换一种结构。

# 第 5 章　卷积网络：把图像先验写进结构

> 图像不是一张“长向量”。它有局部结构，也有平移规律。CNN 的价值，是把这两条先验写进网络。

## 5.1 为什么全连接网络不适合图像

一张 $224 \times 224$ 的 RGB 图片，拉平后是

$$224 \times 224 \times 3 = 150{,}528$$

维向量。如果接一个 1000 个神经元的全连接层，参数量是

$$150{,}528 \times 1000 + 1000 = 150{,}529{,}000 \approx 1.5 \times 10^8.$$

FP32 下，仅这一层权重就约

$$150{,}529{,}000 \times 4\ \text{B} \approx 602\ \text{MB} \approx 0.6\ \text{GB}.$$

这只是第一层。参数多带来三个连锁问题：显存和计算装不下；1.5 亿参数需要海量标注数据才能训好；拉平把二维空间结构变成一维列表，相邻像素的关系被抹平。

一个固定排列的全连接网络当然仍能学习（它可以把任意排列学进去），但它没有任何“局部性”的先验，学起来更慢、更费参数；如果每个样本都随机打乱像素，空间对应关系被彻底破坏，学习会变得极其困难。

图像有两个非常重要的先验：

- **局部性**：一个像素的意义主要由它附近的像素决定。边缘、纹理、角点都是局部模式。
- **平移等变性**：同一个模式出现在图像的不同位置，应该用同一套检测规则去识别。

CNN 就是把这两条先验写进网络结构。

> **CNN 不是“更好的函数”，而是“更好的归纳偏置”**
>
> 从万能逼近的角度看，MLP 和 CNN 都能逼近连续函数。CNN 的优势在于：它把“局部连接、权值共享、平移等变”这些对图像的合理假设写进结构，从而用更少的数据和参数学到更好的解。

## 5.2 局部连接与权值共享

全连接层中，每个输出神经元和所有输入像素相连。卷积层中，每个输出只和输入的一个小窗口（例如 $3 \times 3$）相连。这个窗口就是**卷积核**或**滤波器**。

同一个卷积核在图像的所有位置滑动，使用同一组权重。这意味着：

- 参数量与图像尺寸解耦：一个 $3 \times 3$ 核只有 9 个权重（单通道、无偏置），无论输入是 $28 \times 28$ 还是 $4096 \times 4096$；
- 同一个模式检测器可以在不同位置复用；

- 计算可以写成高度并行的矩阵运算，适合 GPU。

### 平移等变，不是平移不变

这是最容易被说错的一点。设 $T_\delta$ 表示把图像平移 $\delta$，卷积操作为 $*$。在合适的边界条件下，

$$T_\delta(\boldsymbol{X} * \boldsymbol{K}) = (T_\delta \boldsymbol{X}) * \boldsymbol{K}.$$

也就是说：**输入平移，输出也跟着平移**。这叫**平移等变**，不是不变。卷积本身不会把不同位置的响应“压成同一个值”。真正带来一定程度平移不变的是**池化**（尤其是全局池化）和后续的聚合操作：模式在小窗口内移动，最大池化的输出可能不变。

> **“CNN 具有平移不变性”是不准确的**
>
> 卷积是平移**等变**的；池化才提供近似的平移**不变**。而且这种不变性是局部的、有限的：大范围平移、旋转、缩放仍会改变表示。把“等变”和“不变”混为一谈，会导致对 CNN 能力的错误预期。

## 5.3 卷积运算：一次完整手算

深度学习中的“卷积”实际上做的是**互相关**：卷积核在输入上滑动，每个位置做逐元素相乘再求和。严格的数学卷积会把核翻转 $180^\circ$，但框架通常不翻转，因为核是学出来的，翻不翻转不影响表达能力。为简洁，后面仍沿用“卷积”这个习惯叫法。

单通道、步长 1、无填充时：

$$Y_{i,j} = \sum_{u=1}^{k}\sum_{v=1}^{k} X_{i+u-1,\,j+v-1}\,K_{u,v} + b.$$

### 样例：一个竖直边缘检测器

设输入为 $4\times 4$ 矩阵

$$\boldsymbol{X} = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 0 & 1 & 3 & 2 \\ 2 & 0 & 1 & 1 \\ 1 & 1 & 0 & 2 \end{bmatrix}, \qquad \boldsymbol{K} = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}.$$

这个核把左列相加、右列相减，因此对“左亮右暗”的竖直边缘有强响应；如果右亮左暗，响应为负。步长 1、无填充，输出是 $2\times 2$。

左上角位置 $(1,1)$ 的窗口是

$$\begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 0 & 1 \end{bmatrix},$$

计算：

$$Y_{1,1} = 1\cdot 1 + 2\cdot 0 + 0\cdot(-1) + 0\cdot 1 + 1\cdot 0 + 3\cdot(-1) + 2\cdot 1 + 0\cdot 0 + 1\cdot(-1) = -1.$$

右上角位置 $(1,2)$ 的窗口是

$$\begin{bmatrix} 2 & 0 & 1 \\ 1 & 3 & 2 \\ 0 & 1 & 1 \end{bmatrix},$$

计算：

$$Y_{1,2} = 2\cdot 1 + 0\cdot 0 + 1\cdot(-1) + 1\cdot 1 + 3\cdot 0 + 2\cdot(-1) + 0\cdot 1 + 1\cdot 0 + 1\cdot(-1) = -1.$$

类似地，左下角为 $-1$，右下角为 $-3$。最终

$$\boldsymbol{Y} = \begin{bmatrix} -1 & -1 \\ -1 & -3 \end{bmatrix}.$$

输入 $4\times 4$

| | | | |
|---|---|---|---|
| 1 | 2 | 0 | 1 |
| 0 | 1 | 3 | 2 |
| 2 | 0 | 1 | 1 |
| 1 | 1 | 0 | 2 |

$*$

核 $3\times 3$

| | | |
|---|---|---|
| 1 | 0 | −1 |
| 1 | 0 | −1 |
| 1 | 0 | −1 |

$=$

输出 $2\times 2$

| | |
|---|---|
| **−1** | **−1** |
| **−1** | **−3** |

图 5–1　一个竖直边缘检测卷积。核对左上角：$1-0+0-3+2-1=-1$。输出为负，表示窗口内右侧整体比左侧亮。

> **原稿这里的矩阵与公式对不上**
>
> 原稿正文使用的输入矩阵、卷积核，与插图里的矩阵不是同一组；正文把 $Y_{1,2}$ 算成 4，但按它自己给出的数字应为 1，按插图的数字应为 −1。本版统一使用插图中的输入与竖直边缘核，四个输出都重新计算为 $\begin{bmatrix} -1 & -1 \\ -1 & -3 \end{bmatrix}$。

## 输出尺寸

设输入高 $H$、宽 $W$，核大小 $k\times k$，步长 $s$，填充 $p$，膨胀率 $d$（普通卷积 $d=1$），则

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - d(k-1) - 1}{s} \right\rfloor + 1.$$

普通卷积就是

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1.$$

本例：$H=4$，$k=3$，$s=1$，$p=0$，所以 $H_{\text{out}} = (4-3)/1 + 1 = 2$。填充是在输入边缘补零；$p=1$ 时，$3\times 3$ 核可以让 $4\times 4$ 输入保持 $4\times 4$ 输出。

## 多通道与多核

真实输入有多个通道（RGB 是 3 个）。一个卷积核也对应有 $C_{\text{in}}$ 个通道，在每个位置把所有通道的乘加结果再相加：

$$Y_{i,j,o} = \sum_{c=1}^{C_{\text{in}}} \sum_{u=1}^{k} \sum_{v=1}^{k} X_{i+u-1,j+v-1,c}\, K_{u,v,c,o} + b_o.$$

一层通常有 $C_{\text{out}}$ 个卷积核，每个核产生一张特征图，叠起来就是下一层的 $C_{\text{out}}$ 通道输入。参数量为

$$k^2 C_{\text{in}} C_{\text{out}} + C_{\text{out}}.$$

例如 $3 \times 3$、$C_{\text{in}} = 64$、$C_{\text{out}} = 128$：

$$9 \times 64 \times 128 + 128 = 73{,}856.$$

对照 5.1 节 MLP 第一层的 1.5 亿参数，卷积的参数效率高出几个数量级。

| 超参数 | 作用 | 常见取值 | 注意 |
|---|---|---|---|
| 核大小 $k$ | 每个位置看多大范围 | 3、5、7 | 小核堆叠更省参数 |
| 步长 $s$ | 滑动步长 | 1、2 | $s > 1$ 降采样 |
| 填充 $p$ | 边缘补零 | 0、1 | same 保持尺寸 |
| 膨胀 $d$ | 核元素间隔 | 1、2、4 | 不增加参数扩大感受野 |
| 分组 $g$ | 通道分组卷积 | 1、2、… | 减少参数与计算 |
| 通道数 | 特征图数量 | 32、64、128… | 越深通常越多 |

表 5–1　卷积层的主要超参数。设计网络时，输出尺寸、参数量、感受野要一起算。

## 5.4 池化、归一化与经典架构

### 池化

池化在特征图上做局部降采样：最大池化取窗口内最大值，平均池化取平均值，全局池化对整个特征图取最大/平均。$2 \times 2$、步长 2 的最大池化可以把空间尺寸减半。作用包括降低计算量、扩大后续层的感受野、提供一定程度的局部平移不变性。

最大池化的反向传播是一个“路由开关”：梯度只传给前向时取到最大值的那个位置，其余位置为 0。平均池化则把梯度平均分给窗口内所有位置。

### 归一化

深层 CNN 常配合归一化层。最经典的是 BatchNorm：

$$\mu_{\mathcal{B}} = \frac{1}{B}\sum_{i=1}^{B} x_i, \qquad \sigma_{\mathcal{B}}^2 = \frac{1}{B}\sum_{i=1}^{B} (x_i - \mu_{\mathcal{B}})^2,$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \qquad y_i = \gamma \hat{x}_i + \beta.$$

训练时用当前 batch 的均值/方差，推理时用训练阶段累积的滑动平均。BatchNorm 对 batch 大小敏感，分布式训练时还要同步统计量。Transformer 更常用 LayerNorm/RMSNorm，第 8 章会展开。

## 从 LeNet 到 ResNet

**LeNet-5（1998）** 是早期成功落地的卷积网络之一，用于手写数字识别，并曾被用于支票数字读取。它的结构是“卷积—池化—卷积—池化—全连接”，参数量很小，证明了卷积、权值共享、梯度训练可以在真实任务上工作。

**AlexNet（2012）** 在 ImageNet 上取得突破，通常被视为深度学习工业化的起点。它的关键组合是：更深的卷积网络（5 个卷积层 + 3 个全连接层，约 6000 万参数）、ReLU、Dropout、两块 GTX 580 GPU、约 120 万张训练图像。

需要澄清两个常见说法：

1. **“错误率从 26% 降到 16%”是 top-5 错误率。** AlexNet 单模型 top-5 约 18.2%，5 个模型平均约 16.4%，7 个模型集成约 15.3%；当年第二名约 26.2%。不同口径的数字不能混用。
2. **“两块 GTX 580 数据并行”不准确。** 由于单卡只有 3GB 显存，AlexNet 把网络切分到两块 GPU 上（模型并行），只在某些层通信；这不是今天意义上的数据并行。

**VGG（2014）** 的核心洞察是：用多个 $3\times3$ 小核堆叠，代替一个大核。两个 $3\times3$ 卷积的感受野等于一个 $5\times5$，但参数更少：

$$2\times3^2=18<25=5^2,$$

而且中间多了一次非线性。VGG-16/19 结构规整、易于迁移，但参数量较大（VGG-16 约 1.38 亿）。

**ResNet（2015）** 的核心是残差连接：

$$\boldsymbol{y}=\mathcal{F}(\boldsymbol{x})+\boldsymbol{x}.$$

如果恒等映射是最优的，网络只需要把 $\mathcal{F}$ 学成 0，比直接学一个恒等变换容易。残差连接让梯度有一条更直接的路径回传，缓解了深层网络的退化问题。ResNet-152 让 152 层网络可训练，并赢得 ILSVRC 2015。

> **残差连接不只解决“梯度消失”**
>
> 残差连接确实改善了梯度传播，但它更重要的作用是**改变优化问题的参数化**：让网络学习“相对于恒等的修正”，而不是从零学习完整映射。把残差只说成“防止梯度消失”是不完整的。

后续架构中，DenseNet 用密集连接做特征复用，MobileNet 用深度可分离卷积降低计算，EfficientNet 用复合缩放统一调整深度/宽度/分辨率，ConvNeXt 借鉴 Transformer 设计把纯 CNN 现代化，ViT 则把图像切成 patch 用 Transformer 处理。它们不是互相替代的关系，而是不同数据规模与效率约束下的选择。

| 模型 | 年份 | 关键思想 |
| --- | --- | --- |
| LeNet-5 | 1998 | 卷积 + 池化 + 全连接，早期落地 |
| AlexNet | 2012 | ReLU、Dropout、GPU，ImageNet 突破 |
| VGG | 2014 | 小核堆叠，结构规整 |
| Inception | 2014 – 2015 | 多尺度并行、$1\times1$ bottleneck |
| ResNet | 2015 | 残差连接，极深网络可训练 |

| 模型 | 年份 | 关键思想 |
| --- | --- | --- |
| DenseNet | 2017 | 密集连接，特征复用 |
| MobileNet | 2017 | 深度可分离卷积，移动端 |
| EfficientNet | 2019 | 复合缩放 |
| ViT | 2020/2021 | 图像 patch + Transformer |

表 5–2　CNN 家族的代表性模型。年份按论文公开时间标注，实际发表会议可能晚一年。

## 5.5 反向传播与感受野

卷积层本质上是一组共享参数的线性运算，因此可微。反向传播有两个新动作。

### 对输入的梯度

输出位置 $(i,j)$ 的梯度 $\delta_{i,j}$ 会分发给参与该位置乘加的每个输入像素，按卷积核对应位置的权重加权。把所有位置的贡献加起来：

$$\frac{\partial \mathcal{L}}{\partial X_{p,q}} = \sum_{i,j}\sum_{u,v} \delta_{i,j} K_{u,v} \quad （当\ p = i+u-1,\ q = j+v-1）.$$

在 stride 1 时，这等价于把卷积核翻转 180° 后对 $\delta$ 做卷积；一般情形用**转置卷积**或“反向卷积”实现。注意：转置卷积不是卷积的逆运算，它只是反向传播中“把梯度映射回输入形状”的线性操作。

### 对卷积核的梯度

由于核在所有位置共享，总梯度是所有位置梯度之和：

$$\frac{\partial \mathcal{L}}{\partial K_{u,v}} = \sum_{i,j} \delta_{i,j}\, X_{i+u-1,\,j+v-1}.$$

即：核的梯度 = 输出梯度与输入的互相关。这个“共享参数梯度相加”的模式，在 RNN 的时间步共享、嵌入层的查表梯度中都会再次出现。

### 工程实现：im2col + GEMM

直接写多层循环很慢。常见优化是把每个滑动窗口展开成一列，得到 `im2col` 矩阵，然后用一次大矩阵乘（GEMM）完成卷积。代价是额外的内存，但换来了 GPU 的高吞吐。现代框架还会用 Winograd、FFT、直接卷积等算法，根据核大小和硬件选择。

### 感受野

**感受野**是输出特征图上一个点，对应输入图像上的区域大小。对于步长 1 的 $3\times 3$ 卷积：1 层是 $3\times 3$，2 层是 $5\times 5$，$n$ 层是 $(2n+1)\times(2n+1)$。

更一般地，沿网络逐层计算：

$$\mathrm{RF}_l = \mathrm{RF}_{l-1} + (k_l - 1)\cdot d_l \cdot J_{l-1}, \qquad J_l = J_{l-1}\cdot s_l,$$

其中 $J$ 是相邻输出点之间的输入间隔，$d_l$ 是膨胀率，$s_l$ 是步长。初始 $\mathrm{RF}_0 = 1$，$J_0 = 1$。

两个 $3 \times 3$ 堆叠的有效感受野是 $5 \times 5$，但**有效感受野**（对输出实际影响大的区域）通常小于理论感受野，且呈高斯状衰减。不要只看理论值。

## 5.6 CNN 不止用于图像

卷积的思想适用于任何具有局部相关性的网格数据：一维卷积用于文本、音频、传感器时间序列；因果卷积只看过去，不看未来，用于自回归生成；膨胀卷积不增加参数扩大感受野，用于语音、分割；图卷积处理非网格结构（第 9 章）。

CNN 与 Transformer 的取舍，本质上是**归纳偏置 vs 数据规模**的取舍：CNN 的局部性、平移等变是先验，数据少时更省样本；Transformer 的注意力更灵活，数据多时可以学到长距离依赖，但参数和计算更大。实践中常出现混合架构（卷积 stem + Transformer、卷积与注意力交替）。

> **把这一章放回地图**
>
> CNN 用局部连接和权值共享，把图像的先验写进结构。卷积是平移等变的，池化才提供近似不变；残差连接让极深网络可训练。它的参数效率、层级特征和工程实现，深刻影响了后来的视觉与多模态模型。下一章换一种数据结构：序列。

# 第 6 章　循环网络、LSTM 与 GRU：给网络装上记忆

> 序列的关键不是“更长”，而是“顺序本身携带信息”。RNN 的做法，是让每一步的计算依赖上一步的状态。

## 6.1 序列问题：为什么 MLP 和 CNN 不够

语言、日志、股价、音频、用户行为，这些数据的共同点是：顺序重要，长度可变。

“狗咬人”和“人咬狗”词完全相同，意思相反；同一段音频正放和倒放完全不同；日志事件的时间顺序决定故障原因；句子长度从几个 token 到几千个 token 不等。

MLP 要求输入定长、无序；CNN 能看局部窗口，但窗口外的信息看不到（除非堆很多层）。处理序列需要一种新的结构原语：**让网络的计算依赖它自己上一步的状态**。

此外还有参数效率问题：如果为每种序列长度各训一个网络，参数量会爆炸。理想方案是同一个网络在序列上循环使用——这是 CNN 权值共享思想在“时间维度”上的翻版。

## 6.2 简单 RNN：一个带状态的函数

在每个时间步 $t$，RNN 把当前输入 $\boldsymbol{x}_t$ 和上一步隐藏状态 $\boldsymbol{h}_{t-1}$ 拼接（或分别乘矩阵）后，经过线性变换和激活，得到新状态：

$$\boldsymbol{h}_t = \tanh\left(\boldsymbol{W}_h \boldsymbol{h}_{t-1} + \boldsymbol{W}_x \boldsymbol{x}_t + \boldsymbol{b}\right).$$

如果需要输出（例如每个时间步的分类），再加一个输出层：

$$\hat{\boldsymbol{y}}_t = \mathrm{softmax}\left(\boldsymbol{W}_y \boldsymbol{h}_t + \boldsymbol{b}_y\right).$$

三组参数 $\boldsymbol{W}_h$、$\boldsymbol{W}_x$、$\boldsymbol{b}$ 在所有时间步共享。$\boldsymbol{h}_t$ 就是网络的“记忆”：一个固定大小的向量，负责把过去的信息压缩携带到未来。

参数量（单层、输入维度 $m$、隐藏维度 $d$）为

$$d^2 + md + d.$$

### 手算一个标量 RNN

设 $W_h = 0.5$、$W_x = 1$、$b = 0$、$h_0 = 0$，输入序列 $(2, 1, 1)$。标量情形下：

$$h_1 = \tanh(0.5 \times 0 + 1 \times 2) = \tanh(2) \approx 0.9640,$$

$$h_2 = \tanh(0.5 \times 0.9640 + 1) = \tanh(1.4820) \approx 0.9018,$$

$$h_3 = \tanh(0.5 \times 0.9018 + 1) = \tanh(1.4509) \approx 0.8959.$$

可以看到两个现象：早期输入的影响会随着时间步逐步衰减；$h_t$ 被 tanh 压在 $(-1, 1)$ 内，前向数值不会无限增长。但“前向有界”不等于“梯度稳定”。下一节会看到，反向传播的连乘仍然可能消失或爆炸。

> **原稿的 tanh 数值算错了**
>
> 原稿写 $h_1 = \tanh(2) \approx 0.76$，但 $\tanh(2) \approx 0.9640$；后续的 0.88、0.89 也随之错误。本版重新计算为 0.9640、0.9018、0.8959。手算例子必须能复核，否则读者会对“记忆衰减”的量级产生误解。

### 展开计算图

把 RNN 沿时间展开，它等价于一个深度等于序列长度的网络，但每层共享参数：

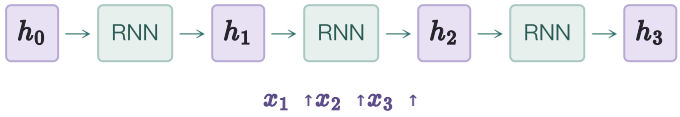



图 6–1　RNN 沿时间展开。每个时间步使用同一组参数；反向传播要穿过整条链。

## 6.3 时间反向传播：梯度如何穿过时间

训练 RNN 仍然用链式法则，只是链条沿时间变长，称为**沿时间反向传播**（BPTT）。

设损失只在最后一步计算（或对各步损失求和），要计算 $\partial \mathcal{L}/\partial \boldsymbol{h}_t$，需要穿过中间每一步：

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_T} \prod_{k=t+1}^{T} \frac{\partial \boldsymbol{h}_k}{\partial \boldsymbol{h}_{k-1}}.$$

标量情形下，

$$\frac{\partial h_k}{\partial h_{k-1}} = W_h \tanh'(z_k) = W_h \left(1 - h_k^2\right).$$

矩阵情形下，雅可比是

$$\frac{\partial \boldsymbol{h}_k}{\partial \boldsymbol{h}_{k-1}} = \operatorname{diag}\left(1 - \boldsymbol{h}_k^2\right) \boldsymbol{W}_h.$$

共享参数的梯度是所有时间步梯度之和。令列向量 $\boldsymbol{\delta}_t = \partial \mathcal{L}/\partial \boldsymbol{h}_t$，则

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}_h} = \sum_{t=1}^{T} \left(\boldsymbol{\delta}_t \odot \left(1 - \boldsymbol{h}_t^2\right)\right) \boldsymbol{h}_{t-1}^{\top}.$$

工程实现时，框架沿时间展开计算图，缓存每步的 $\boldsymbol{h}_t$ 和中间值，反向时从 $T$ 走到 1。内存开销与序列长度成正比——这是长序列训练的第一座大山。实践中常用**截断** BPTT：每 $K$ 步切断梯度链，只反传到最近的 $K$ 步。它牺牲了长程信用分配，换取了可控的内存和计算。

## 6.4 梯度消失与爆炸：连乘的代价

设 $W_h = 0.5$，且 $\tanh'$ 最大为 1，则每步因子的绝对值不超过 $0.5$。回传 10 步：

$$0.5^{10} \approx 0.00098.$$

回传 50 步：

$$0.5^{50} \approx 8.9 \times 10^{-16}.$$

在 FP16 中这已经下溢；在 FP32 中虽然仍可表示，但相对于初始梯度已微不足道。梯度实际上“消失”了。后果是：第 1 个词对第 100 步的损失几乎没有影响，RNN 的“记忆”在实践中常常只有几十步，具体取决于权重、激活和数据。

反过来，若 $W_h = 2$，则 $2^{10} = 1024$，$2^{50} \approx 1.1 \times 10^{15}$，梯度爆炸，训练直接发散。

### 数学本质：谱半径

连乘 $\prod_k W_h \operatorname{diag}(1 - h_k^2)$ 的行为，主要由 $W_h$ 的**谱半径**（最大特征值的绝对值）决定：谱半径小于 1，梯度指数衰减；大于 1，梯度指数爆炸；接近 1，梯度可以传播更远，但也更容易不稳定。

想让梯度稳定穿过 100 步，需要连乘的整体尺度在 1 附近。这不能靠“碰巧初始化”可靠实现；需要结构上的设计（门控、残差）或训练技巧（正交初始化、梯度裁剪）。

工程上的止血贴：

- **梯度裁剪**：当梯度范数超过阈值时按比例缩放，主要治爆炸；
- **梯度范数监控**：训练日志里必看，突然变大往往是发散前兆；
- **更好的初始化**：正交初始化、单位矩阵初始化；
- **门控结构**：LSTM/GRU 用加性路径绕开部分连乘；
- **残差连接**：给梯度一条更短的路径。

> **前向有界 ≠ 梯度稳定**
>
> tanh 把 $h_t$ 压在 $(-1, 1)$，所以前向状态不会数值爆炸；但反向传播乘的是 $W_h \tanh'$，$W_h$ 可以大于 1，$\tanh'$ 也可以接近 0。前向稳定和梯度稳定是两个问题。

## 6.5 LSTM：用门控修一条加性通道

### 历史

LSTM 由 Hochreiter 和 Schmidhuber 于 1997 年提出。原始 LSTM 有输入门和输出门，核心是**常量误差传送带**：当门打开时，细胞状态的误差可以近乎恒定地流动。

今天常说的“标准 LSTM”还包含**遗忘门**，这是 Gers、Schmidhuber 和 Cummins 在 2000 年加入的。遗忘门让网络可以主动丢弃旧信息，对连续预测任务很关键。把 1997 年的 LSTM 和 2000 年的 LSTM 混为一谈，是常见的历史错误。

### 公式

在每个时间步 $t$，给定输入 $x_t$ 和上一步隐藏状态 $h_{t-1}$：

$$f_t = \sigma\left(W_f [h_{t-1}, x_t] + b_f\right),$$

$$i_t = \sigma\left(W_i [h_{t-1}, x_t] + b_i\right),$$

$$\tilde{\boldsymbol{c}}_t = \tanh\left(\boldsymbol{W}_c[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_c\right),$$

$$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{c}}_t,$$

$$\boldsymbol{o}_t = \sigma\left(\boldsymbol{W}_o[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_o\right), \qquad \boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t).$$

遗忘门控制旧记忆保留多少，输入门控制新信息写入多少，输出门控制当前暴露多少。四个门/候选各有一组权重，因此 LSTM 的参数量大约是简单 RNN 的 4 倍（忽略偏置和 peephole）：

$$4(d^2 + md + d).$$

### 为什么加性路径有帮助

简单 RNN 的状态更新是

$$\boldsymbol{h}_t = \tanh(\boldsymbol{W}_h \boldsymbol{h}_{t-1} + \cdots),$$

梯度必须穿过 $\boldsymbol{W}_h$ 和 tanh 的导数。LSTM 的细胞状态是

$$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \cdots,$$

因此

$$\frac{\partial \boldsymbol{c}_t}{\partial \boldsymbol{c}_{t-1}} = \operatorname{diag}(\boldsymbol{f}_t).$$

如果遗忘门学到 $\boldsymbol{f}_t \approx \boldsymbol{1}$，梯度沿细胞状态几乎无损地直通。这条**加性路径**比反复乘矩阵更容易保持长程信息。

但要注意：$\boldsymbol{f}_t$ 是学出来的，不保证等于 1；如果它小于 1，梯度仍会衰减。输出门、tanh、输入门仍然引入非线性，其他路径仍可能消失或爆炸。LSTM 是**缓解**长依赖问题，不是彻底解决。

> **“LSTM 有一条导数恒为 1 的恒等路径”是过度简化**
>
> 只有当遗忘门接近 1、且只考虑细胞状态这条路径时，导数才接近 1。实际网络中，遗忘门可以学到接近 0，梯度仍会消失；而且梯度还要穿过输出门和后续层。正确说法是：LSTM 提供了一条**更容易**保持梯度的加性通道，而不是一条永远恒等的通道。

## 6.6 GRU：更少的门，类似的思想

GRU 由 Cho 等人在 2014 年提出。它把 LSTM 的输入门和遗忘门合并成一个**更新门**，并引入**重置门**，没有独立的细胞状态。

更新门：

$$\boldsymbol{z}_t = \sigma\left(\boldsymbol{W}_z[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_z\right).$$

重置门：

$$\boldsymbol{r}_t = \sigma\left(\boldsymbol{W}_r[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_r\right).$$

候选状态：

$$\tilde{\boldsymbol{h}}_t = \tanh\left(\boldsymbol{W}_h[\boldsymbol{r}_t \odot \boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_h\right).$$

更新：

$$\boldsymbol{h}_t = (1 - \boldsymbol{z}_t) \odot \boldsymbol{h}_{t-1} + \boldsymbol{z}_t \odot \tilde{\boldsymbol{h}}_t.$$

GRU 的参数和计算通常比 LSTM 少，在很多任务上表现相当。它同样使用加性更新来缓解梯度问题。

| 特性 | 简单 RNN | LSTM | GRU |
|---|---|---|---|
| 状态 | 隐藏状态 | 隐藏状态 + 细胞状态 | 隐藏状态 |
| 门 | 无 | 输入、遗忘、输出 | 更新、重置 |
| 参数量（相对） | 1× | 约 4× | 约 3× |
| 长依赖 | 弱 | 较强 | 较强 |
| 时间步并行 | 否 | 否 | 否 |

表 6–1　RNN 家族对比。参数量是数量级估计，实际取决于实现与是否使用 peephole、投影层等。

## 6.7 结构性瓶颈与遗产

即便 LSTM 缓解了梯度问题，RNN 家族仍有三个结构性缺陷。

**第一，无法在时间维度并行。** $\boldsymbol{h}_t = \boldsymbol{f}(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t)$ 意味着第 $t$ 步必须等第 $t-1$ 步算完。1000 个 token 就要串行 1000 步，GPU 的并行能力用不上。

**第二，固定大小的状态瓶颈。** 无论句子多长，全部历史必须压进一个固定维度的 $\boldsymbol{h}_T$。长句子的早期信息必然被压缩、覆盖或丢失。Seq2Seq 只把这个状态传给解码器，瓶颈更严重（第 7 章）。

**第三，长程信用分配仍然困难。** LSTM 的加性路径有帮助，但梯度仍要穿过很多步；门控也可能学会“尽早丢弃”。超长依赖仍然很难。

> **KV Cache 不是“RNN 状态压缩”的直系后代**
>
> RNN 把历史压缩成一个固定大小的状态；Transformer 的 KV Cache 则**显式保存**过去所有 token 的 Key/Value，内存随上下文长度线性增长。两者解决的问题相似（避免重复计算历史），但机制相反：一个是“压缩”，一个是“缓存”。

### RNN 仍然有用的场景

流式/在线推理每来一个 token 只更新一次状态，延迟低、内存恒定；边缘设备模型小、功耗低；短序列时间序列（传感器、金融、工业监控）仍然常用；在某些低资源任务中，RNN 作为编码器仍有竞争力。架构没有绝对优劣，只有与约束的匹配。

**把这一章放回地图**

RNN 用共享参数和时间递归处理变长序列；BPTT 的连乘导致梯度消失/爆炸；LSTM/GRU 用门控和加性路径缓解，但不能根治；固定状态和时间串行是它的结构性上限。下一章的注意力，就是绕开这两个上限的一种方法。

# 第 7 章　Seq2Seq 与注意力：Transformer 的前夜

> 编码器—解码器解决了“变长到变长”，但留下一个固定向量瓶颈。注意力让解码器按需回看，而不是只接收一次摘要。

## 7.1 序列到序列：输入输出都可以变长

很多任务不是“输入一个向量，输出一个标签”，而是“输入一个序列，输出另一个序列”：机器翻译、文本摘要、对话、语音识别、代码生成。

这些任务的输入长度和输出长度都不固定，而且输入与输出之间往往没有一一对应的位置关系。2014 年前后，Seq2Seq 框架成为神经机器翻译的主流：用一个 RNN 读入源序列，再用另一个 RNN 生成目标序列。

### 编码器

编码器逐步读入源序列 $\boldsymbol{x}_1, \dots, \boldsymbol{x}_T$：

$$\boldsymbol{h}_t^{\text{enc}} = \text{RNN}_{\text{enc}}\left(\boldsymbol{h}_{t-1}^{\text{enc}}, \boldsymbol{x}_t\right).$$

最后一步的隐藏状态 $\boldsymbol{h}_T^{\text{enc}}$ 被视为整个源句子的“语义向量”：

$$\boldsymbol{c} = \boldsymbol{h}_T^{\text{enc}}.$$

### 解码器

解码器以 $\boldsymbol{c}$ 为初始状态，逐步生成目标序列：

$$\boldsymbol{s}_t = \text{RNN}_{\text{dec}}\left(\boldsymbol{s}_{t-1}, \boldsymbol{y}_{t-1}, \boldsymbol{c}\right),$$

$$p(\boldsymbol{y}_t \mid \boldsymbol{y}_{<t}, \boldsymbol{x}) = \text{softmax}\left(\boldsymbol{W}_o \boldsymbol{s}_t + \boldsymbol{b}_o\right).$$

训练时，$\boldsymbol{y}_{t-1}$ 通常是**真实的前一个 token**（teacher forcing）；推理时，用模型自己上一步生成的 token（free-running）。两者不一致会导致**暴露偏差**：训练时没见过自己的错误，推理时错误会累积。缓解方法包括 scheduled sampling、强化学习、beam search 等。

一个历史细节：Sutskever 等（2014）发现，把源句子**倒序**输入编码器，能显著改善翻译效果。直觉是：源句开头与目标句开头的距离被拉近，优化更容易。这个技巧不是架构的本质，但它说明序列顺序、优化难度和数据对齐之间有微妙关系。

### 解码策略

贪心解码每步取概率最大的 token；束搜索保留 $k$ 个候选序列，平衡全局概率；采样按概率采样，配合温度、top-k、nucleus sampling 增加多样性；长度惩罚避免束搜索偏向过短或过长。

## 7.2 定长向量瓶颈

Seq2Seq 的致命弱点是：整个源句子的信息，必须压缩进一个固定维度的向量 $\boldsymbol{c}$。

打个比方：让你读完一篇 500 词的文章，只许写下一个 100 字的摘要，再要求译者仅凭摘要全文复现。短句还可以，长句必然丢信息。

量化一下：若 $\boldsymbol{h_T}$ 是 512 维 FP32 向量，大小是 $\mathbf{512 \times 4 = 2048}$ 字节，约 2 KB；一篇 500 词的英文原文约 3 KB 文本。但“文本字节”与“语义容量”不是同一回事，不能简单比较。关键在于：固定维度向量的容量有限，而长句的语义关系数量随长度增长；压缩损失会随长度增加。

更严重的是，$\boldsymbol{h_T}$ 还要经过 tanh 逐元素压缩，并受 RNN 梯度消失的影响。长序列的早期信息本来就难以保留。Bahdanau 等的实验图显示：基线 Seq2Seq 的 BLEU 随句子变长急剧下降，而带注意力的模型在长句上稳定得多。以论文 Figure 2 为例，RNNenc-30 在句子长度约 10 时 BLEU 约 20，到长度 50 时已降到个位数；RNNsearch-50 则大致保持在 26 左右（读图近似值，具体以原图为准）。

> **原稿的 BLEU 数字不准确**
>
> 原稿写“句子长度从 10 词增到 50 词，BLEU 从约 35 暴跌到 17”。Bahdanau 论文 Figure 2 中，基线 RNNenc-30 在长度 10 附近约为 20，长度 50 附近已降到 2 – 3；带注意力的 RNNsearch-50 约为 26。“35→17”不符合原图。本版改用定性描述 + 读图近似，并明确标注。

### 瓶颈不在“RNN 记不住”，而在“只准用一个状态通信”

有人把 Seq2Seq 的失败完全归因于 RNN 的记忆容量。更准确的说法是：编码器最终只把最后一个状态交给解码器，这条通信信道太窄。编码器的中间状态其实包含丰富的局部信息，但解码器看不到。注意力的核心思想就是：**让解码器按需查询编码器的所有状态，而不是只接收一次摘要。**

## 7.3 Bahdanau 注意力：按需查询

Bahdanau、Cho 和 Bengio（2014，ICLR 2015）在 Seq2Seq 上加入了一个轻量但革命性的改动：解码器每生成一个词，就回看编码器的所有隐藏状态，计算一个加权平均作为上下文向量。

设编码器状态为 $\boldsymbol{h_1, \dots, h_T}$，解码器上一步状态为 $\boldsymbol{s_{t-1}}$。

打分：

$$e_{t,j} = \boldsymbol{v_a}^{\top} \tanh\left(\boldsymbol{W_a} \boldsymbol{s_{t-1}} + \boldsymbol{U_a} \boldsymbol{h_j}\right).$$

这里 $a(\cdot,\cdot)$ 是一个小型前馈网络（加性注意力）。$\boldsymbol{v_a}$、$\boldsymbol{W_a}$、$\boldsymbol{U_a}$ 都是可学习参数。

归一化：

$$\alpha_{t,j} = \frac{\exp(e_{t,j})}{\sum_{k=1}^{T} \exp(e_{t,k})}.$$

加权和：

$$\boldsymbol{c_t} = \sum_{j=1}^{T} \alpha_{t,j} \boldsymbol{h_j}.$$

解码：

$$s_t = \mathrm{RNN}_{\mathrm{dec}}\left(s_{t-1}, [y_{t-1}; c_t]\right).$$

$\alpha_{t,j}$ 可以理解为“生成第 $t$ 个目标词时，第 $j$ 个源词的重要性”。把所有 $\alpha$ 画成矩阵，就是一张**软对齐图**。

## 手算一个注意力步

设编码器有三个状态：

$$h_1 = (1, 0), \qquad h_2 = (1, 0), \qquad h_3 = (1, 1).$$

打分已经算出：

$$e_1 = 1, \qquad e_2 = 1, \qquad e_3 = 2.$$

先算指数：

$$e^1 \approx 2.718, \qquad e^2 \approx 2.718, \qquad e^3 \approx 7.389.$$

分母：

$$2.718 + 2.718 + 7.389 = 12.825.$$

权重：

$$\alpha_1 = \alpha_2 = \frac{2.718}{12.825} \approx 0.212, \qquad \alpha_3 = \frac{7.389}{12.825} \approx 0.576.$$

上下文向量：

$$c = 0.212(1,0) + 0.212(1,0) + 0.576(1,1) \approx (1.000, 0.576).$$

解读：解码器这一步主要“盯着”第 3 个源词（权重 0.576），同时兼顾前两个词。权重随解码进度每步重新计算——这就是“注意力”的含义：动态决定看哪里。

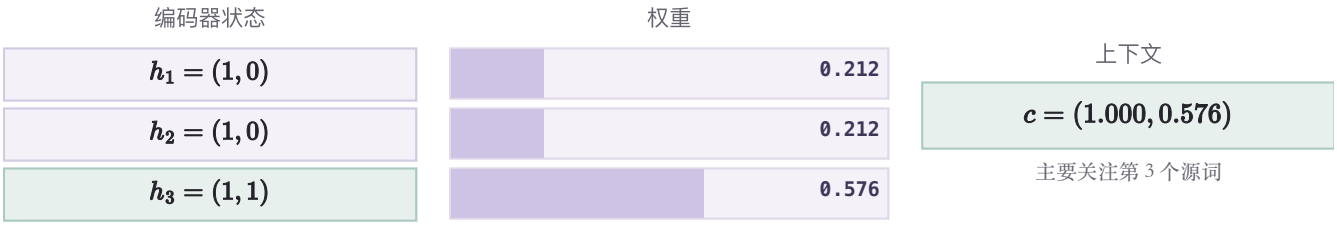



图 7–1　Bahdanau 注意力的一个解码步。权重由 softmax 归一化，上下文是编码器状态的加权和。

## 加性注意力与乘性注意力

Bahdanau 的注意力是**加性**的：先把 query 和 key 投影、相加、过 tanh，再用一个向量打分。Luong 等（2015）提出**乘性注意力**：

$$\mathrm{score}(s_t, h_j) = \begin{cases} s_t^\top h_j, & \text{dot}, \\ s_t^\top W_a h_j, & \text{general}, \\ \cos(s_t, h_j), & \text{cosine}. \end{cases}$$

乘性注意力可以用矩阵乘高效实现，后来成为 Transformer 缩放点积注意力的直接前身。

–

## 7.4 从注意力到 Transformer：Q/K/V 的软字典

### 一个直觉：注意力是“软检索”

把注意力想象成一个字典查询：

- **Query（Q）**：我想找什么；
- **Key（K）**：每个条目的索引/标签；
- **Value（V）**：每个条目的内容。

Query 和每个 Key 做匹配，得到分数；softmax 把分数变成权重；再对 Value 加权求和，得到检索结果。与硬检索“只取一个条目”不同，注意力是**软检索**：所有条目都按权重贡献一点。

在 Transformer 中，Q/K/V 都是输入经过可学习线性投影得到的：

$$\boldsymbol{Q}=\boldsymbol{X}\boldsymbol{W}_Q,\qquad \boldsymbol{K}=\boldsymbol{X}\boldsymbol{W}_K,\qquad \boldsymbol{V}=\boldsymbol{X}\boldsymbol{W}_V.$$

这就是“Q/K/V 投影矩阵可学习”的含义：模型自己学习“用什么问、用什么索引、取什么内容”。

### 缩放点积注意力

$$\mathbf{Attention}(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V})=\mathbf{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\top}}{\sqrt{d_k}}\right)\boldsymbol{V}.$$

其中 $d_k$ 是每个 head 的 key 维度。除以 $\sqrt{d_k}$ 的原因：若 $q_i,k_i$ 独立、均值 0、方差 1，则点积 $q^{\top}k$ 的方差约为 $d_k$；不缩放时，softmax 的输入方差随维度增大，容易饱和（一个位置权重接近 1，其余接近 0），梯度变小。

### 自注意力、交叉注意力与掩码

- **自注意力**：Q/K/V 都来自同一个序列，每个位置可以关注同一序列的所有位置。
- **交叉注意力**：Q 来自解码器，K/V 来自编码器；用于 Seq2Seq、多模态。
- **因果掩码**：在自回归生成中，位置 $t$ 只能看 $\le t$ 的位置，不能看未来。实现时把未来位置的分数设为 $-\infty$，softmax 后权重为 0。
- **填充掩码**：忽略 padding 位置。

### 多头注意力

单个注意力头只能学习一种“相关性模式”。多头注意力并行做 $H$ 次不同的投影：

$$\mathbf{head}_i=\mathbf{Attention}\left(\boldsymbol{Q}\boldsymbol{W}_Q^{(i)},\boldsymbol{K}\boldsymbol{W}_K^{(i)},\boldsymbol{V}\boldsymbol{W}_V^{(i)}\right),$$

$$\mathbf{MultiHead}(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V})=\mathbf{Concat}(\mathbf{head}_1,\dots,\mathbf{head}_H)\boldsymbol{W}_O.$$

每个头在低维子空间中关注不同的关系（语法、指代、位置等），最后拼接并通过输出投影融合。多头不是简单的“集成”，而是让模型在多个表示子空间里并行检索。

### Transformer 块

一个典型的 Transformer 层（pre-LN 版本）是：

$$\boldsymbol{x} \leftarrow \boldsymbol{x} + \mathbf{MHSA}\left(\mathbf{LN}(\boldsymbol{x})\right),$$

$$\boldsymbol{x} \leftarrow \boldsymbol{x} + \mathbf{FFN}\left(\mathbf{LN}(\boldsymbol{x})\right).$$

其中 LN 是层归一化，MHSA 是多头自注意力，FFN 是逐位置前馈网络，通常是两层 MLP 或门控 FFN，残差连接包在每个子层外面。

原始 Transformer 使用 post-LN（先残差、再归一化）；现代大模型常用 pre-LN 或 RMSNorm，因为训练更稳定。不同实现的细节不同，但"注意力 + FFN + 残差 + 归一化"的骨架是一致的。

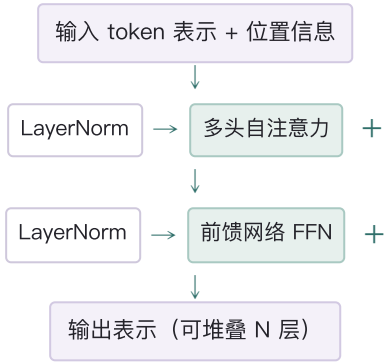



图 7–2　Pre–LN Transformer 块。残差连接让梯度有更短的路径；FFN 逐位置处理，注意力负责混合 token 信息。

### 为什么 Transformer 能取代 RNN

1. **并行**：自注意力可以一次计算所有位置，训练时不受时间步串行限制；
2. **短路径**：单层内任意两个位置直接相连，长距离依赖的梯度路径比 RNN 的 $O(T)$ 短；
3. **表达灵活**：注意力按内容动态路由，而不是固定局部窗口或固定状态；
4. **可扩展**：矩阵乘友好，适合 GPU/TPU 和大规模并行训练。

但 Transformer 不是没有代价：二次复杂度（序列长度 $n$ 时，注意力分数矩阵是 $n \times n$，时间和显存随 $n^2$ 增长）；数据饥饿（缺少 CNN/RNN 的强先验，需要更多数据或预训练）；必须显式注入位置编码；自回归生成需要 KV Cache，显存随上下文增长。

> **"Transformer = 去掉递归 + 自注意力 + 工程细节"是简化说法**
>
> 这个说法有助于记忆，但容易漏掉关键组件：多头注意力、残差连接、层归一化、逐位置 FFN、位置编码、掩码、训练策略。真正让 Transformer 工作的，是这些组件的组合，而不是单独一个注意力公式。

## 7.5 长上下文与可解释性

标准自注意力的 $O(n^2)$ 成本是长上下文的主要障碍。常见改进：稀疏注意力只关注局部窗口或少量全局 token；滑动窗口只关注最近 $w$ 个位置，适合流式/长文本；线性注意力用核函数近似 softmax，把复杂度降到 $O(n)$；低秩/随机特征方法（Linformer、Performer）；FlashAttention 不改变数学结果，通过分块和重计算减少显存访问；MQA/GQA 让多个 query head 共享 key/value head，减少 KV Cache；RoPE/ALiBi 等位置方案改善长度外推。

| 方法 | 复杂度 | 主要收益 | 代价 |
| --- | --- | --- | --- |
| 标准注意力 | $O(n^2 d)$ | 表达力强、实现简单 | 长序列贵 |
| 滑动窗口 | $O(nwd)$ | 局部高效、流式友好 | 全局信息需额外机制 |
| 稀疏/全局 token | 亚二次 | 兼顾局部与全局 | 模式需设计 |
| 线性注意力 | $O(nd^2)$ | 长序列可扩展 | 近似误差、表达力下降 |
| FlashAttention | $O(n^2 d)$ 计算，显存更省 | 实际速度/显存优化 | 不改变渐近复杂度 |
| MQA/GQA | 同标准 | KV Cache 更小 | 可能损失表达力 |

表 7–1　常见长上下文注意力优化。复杂度是大致量级，实际取决于实现、head 数和硬件。

## 注意力权重不是解释

注意力权重常被画成热力图，用来“解释”模型在看哪里。这有一定参考价值，但不能直接当作因果解释：权重高不等于因果重要；不同 head 的权重可能互相补偿；softmax 权重受分数尺度影响；注意力图是“模型内部的检索模式”，不是人类语义解释。更严谨的解释需要消融、梯度归因、因果干预等实验。

> **把这一章放回地图**
>
> Seq2Seq 用编码器—解码器处理变长序列，最后隐藏状态形成定长瓶颈；Bahdanau 注意力让解码器按需查询编码器所有状态；Transformer 把加性注意力换成缩放点积，加入多头、自注意力、残差、归一化、FFN 和位置编码。下一章把这些模型放进训练系统。

# 第 8 章　训练工程：从一行梯度下降到分布式系统

> 模型是数学，训练是系统。损失不降，未必是模型错了；也可能是数据、数值、显存或通信出了问题。

## 8.1 一个训练循环包含什么

第 3 章的手算展示了训练的本质：前向算损失，反向算梯度，沿负梯度更新，再来一遍。真实训练在此基础上增加了数据管线、优化器、混合精度、分布式、监控和评估：

| | |
|---|---|
| 数据 | 采集 / 清洗 / 去重 / 分词 / 打包 / 打乱 / 增强 |
| 模型 | 层、参数、前向、激活、损失 |
| 反向 | 自动微分、梯度、裁剪、累积 |
| 优化 | SGD / AdamW、学习率调度、权重衰减 |
| 系统 | 混合精度、显存、分布式、检查点、监控 |
| 评估 | 验证集、指标、校准、人工评估、回归测试 |

图 8–1　训练系统的六层。任何一层出问题，损失曲线都可能“看起来正常”，但模型实际没有学好。

数据决定模型的上限。去重、过滤低质量内容、检查测试集污染、配好多语言/多领域比例、保证标注一致性，往往比换一个模型结构更重要。批处理影响梯度噪声和吞吐；shuffle 避免顺序偏差；prefetch 让 GPU 不等数据；packing 把多条短序列拼成一个长序列，减少 padding 浪费，但需要正确的 attention mask。

数据增强是注入“标签不变”先验：图像裁剪、翻转、颜色抖动、Mixup、CutMix；文本回译、同义替换（需谨慎）；音频加噪、变速、SpecAugment。如果增强改变了标签，就会伤害训练。

## 8.2 损失、优化器与学习率

分类用交叉熵，回归用 MSE/MAE/Huber，序列生成用 token 级交叉熵，对比学习用 InfoNCE，排序用 pairwise/listwise，对齐用偏好损失（如 DPO）。

**标签平滑**把 one-hot 目标改成

$$y_i^{\mathrm{LS}} = (1-\epsilon)y_i + \frac{\epsilon}{K},$$

可以缓解过度自信、改善校准，但在某些任务上会损害精度。

大模型常用 AdamW：一阶矩、二阶矩自适应，解耦权重衰减，需要偏差修正。超参数通常 $\beta_1 = 0.9$、$\beta_2 = 0.95$ 或 $0.999$、$\epsilon = 10^{-8}$。小模型/卷积网络也常用 SGD + Momentum，泛化有时更好。没有“万能最优优化器”。

学习率调度常见 warmup + cosine/linear decay。warmup 从小学习率逐渐升高，避免训练初期不稳定；cosine 平滑降到接近 0；step decay 按里程碑下降。峰值学习率、warmup 步数、最终学习率都是重要超参数。

**梯度裁剪**按全局范数缩放：

$$g \leftarrow g \cdot \min\left(1, \frac{\tau}{\|g\|_2}\right),$$

防止单步更新过大，尤其在 RNN、Transformer 和混合精度训练中常用。裁剪不等于解决梯度消失，它只控制爆炸。

## 8.3 初始化、归一化与正则化

初始化要打破对称性并控制方差。Xavier 适合 tanh/sigmoid，He 适合 ReLU；残差分支的缩放要与架构匹配；embedding 常用小方差正态；归一化层的 $\gamma$ 初始化为 1，$\beta$ 为 0。

**BatchNorm** 对每个通道统计 batch 均值和方差，训练时更新滑动平均，推理时使用滑动平均。优点：加速收敛、允许更大学习率；缺点：依赖 batch 大小、分布式训练要同步统计量、序列模型不自然。

**LayerNorm** 对每个样本的所有特征归一化，不依赖 batch：

$$\mu = \frac{1}{D}\sum_{i=1}^{D} x_i, \qquad \sigma^2 = \frac{1}{D}\sum_{i=1}^{D}(x_i-\mu)^2,$$

$$y_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i.$$

Transformer 常用 LayerNorm，因为它对 batch 大小不敏感，适合变长序列。**RMSNorm** 不减均值，只除以均方根：

$$y_i = \gamma_i \frac{x_i}{\sqrt{\frac{1}{D}\sum_{j=1}^{D} x_j^2 + \epsilon}},$$

计算更省，在现代大模型中常见。

Pre-LN 写成 $x \leftarrow x + \mathrm{Sublayer}(\mathrm{LN}(x))$，Post-LN 写成 $x \leftarrow \mathrm{LN}(x + \mathrm{Sublayer}(x))$。现代大模型更常用 Pre-LN 或 RMSNorm，训练更稳定。

正则化手段包括：权重衰减限制大权重；Dropout 训练时随机置零激活，推理时关闭，inverted dropout 会按 $1/(1-p)$ 缩放；早停；数据增强；标签平滑；随机深度；Mixup/CutMix。正则化不是越多越好：强正则化会欠拟合，Dropout 与 BatchNorm 一起用有时会冲突，权重衰减通常不适用于 LayerNorm 参数和偏置。

## 8.4 混合精度与显存

FP16 速度快、显存省，但动态范围小，梯度容易下溢/溢出，通常需要 loss scaling；BF16 动态范围接近 FP32，精度较低，大模型训练常用，通常不需要 loss scaling；FP8 需要更精细的缩放策略。

典型混合精度训练：用 FP16/BF16 做前向和矩阵乘；用 FP32 保存主权重和累加；反向得到 FP16/BF16 梯度；转成 FP32 更新主权重；FP16 时用 loss scaling 防止梯度下溢。

训练显存大致包括：参数、梯度、优化器状态、激活、临时缓冲区。Adam + 混合精度训练每参数约需十几到二十几字节；一个 7B 模型，仅参数、梯度、优化器状态就可能超过 100 GB。这就是分布式训练、ZeRO/FSDP、激活检查点等技术存在的原因。

- **梯度累积**：多个 micro-batch 累加后再更新，模拟更大 batch；

- **激活检查点**：前向只保存部分激活，反向时重算，用计算换显存；
- **ZeRO/FSDP**：把参数、梯度、优化器状态分片到多卡，降低单卡显存。

## 8.5 分布式：切什么、通信什么

| 策略 | 切分对象 | 主要通信 | 适用 |
| --- | --- | --- | --- |
| 数据并行（DP/DDP） | batch | 梯度 all-reduce | 模型能放进单卡 |
| 张量并行（TP） | 层内矩阵 | 每层激活 all-reduce | 单层太大 |
| 流水线并行（PP） | 层间 | 阶段间激活传递 | 深模型、多机 |
| 序列并行 | 序列维度 | 注意力/归一化通信 | 长序列 |
| FSDP/ZeRO | 参数/梯度/优化器状态 | 分片通信 | 显存受限 |
| 专家并行（MoE） | 专家 | token 路由 all-to-all | 稀疏大模型 |

表 8–1　常见并行策略。实际训练常组合多种并行（如 TP + PP + DP）。

数据并行最简单：每张卡复制模型，处理不同 batch，反向后的梯度做 all-reduce 平均。张量并行把矩阵乘按行/列切开，适合单层参数放不下的情况；流水线并行把不同层放到不同设备，用 micro-batch 填充流水线；FSDP/ZeRO 则把训练状态分片，按需 gather。核心矛盾是计算与通信的重叠：通信太慢，GPU 就会空等。

## 8.6 调试与评估

训练不收敛时，按顺序排查：

1. **数据**：标签对不对？输入归一化了吗？有没有 NaN/Inf？batch 里是否全是 padding？
2. **模型**：形状对不对？输出维度、mask、残差连接、归一化位置是否正确？
3. **损失**：损失函数与输出激活是否匹配？标签范围是否正确？
4. **梯度**：有没有梯度？梯度范数多大？是否爆炸/消失？做梯度检查。
5. **优化器**：学习率是否太大/太小？warmup 是否足够？权重衰减是否过强？
6. **精度**：FP16 是否溢出？loss scale 是否合适？是否需要 BF16？
7. **过拟合小批量**：取一个 batch，关闭正则化，训练到接近 0 损失；如果做不到，说明模型/损失/优化器有 bug。
8. **对照实验**：一次只改一个变量；记录配置、种子、数据版本。

评估指标因任务而异：分类看准确率、精确率/召回率、F1、AUC；回归看 MSE/MAE/$R^2$；生成看 BLEU/ROUGE/METEOR/BERTScore 和人工评估；语言模型看困惑度、下游任务准确率、校准、幻觉率；检索看 Recall@k、MRR、NDCG；对齐看胜率、偏好、安全性、拒答率。

指标有几个常见陷阱：类别不平衡时准确率会骗人；测试集污染会让指标虚高；BLEU 高不等于翻译好；小测试集上的微小差异可能是噪声；人工评估受标注者、提示词、顺序影响。

## 8.7 手写一个可训练的 MLP

下面是一个完整的 NumPy 两层 MLP，训练 XOR。它对应第 3 章的公式，但把四个参数都更新，并修正了原稿的参数计数错误。

```
import numpy as np

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
Y = np.array([[0], [1], [1], [0]], dtype=float)

rng = np.random.default_rng(0)
W1 = rng.normal(0, np.sqrt(2 / 2), (2, 4))   # He 初始化
b1 = np.zeros((1, 4))
W2 = rng.normal(0, np.sqrt(2 / 4), (4, 1))
b2 = np.zeros((1, 1))

def relu(z):
    return np.maximum(0, z)

for step in range(5001):
    # 前向
    z1 = X @ W1 + b1          # (4,2) @ (2,4) -> (4,4)
    h = relu(z1)              # (4,4)
    y = h @ W2 + b2           # (4,4) @ (4,1) -> (4,1)
    loss = ((y - Y) ** 2).mean()

    # 反向
    dY = 2 * (y - Y) / 4
    dW2 = h.T @ dY
    db2 = dY.sum(axis=0, keepdims=True)
    dh = dY @ W2.T
    dz1 = dh * (z1 > 0)
    dW1 = X.T @ dz1
    db1 = dz1.sum(axis=0, keepdims=True)

    # 更新
    lr = 0.1
    W2 -= lr * dW2
    b2 -= lr * db2
    W1 -= lr * dW1
    b1 -= lr * db1

    if step % 1000 == 0:
        print(f"step {step}, loss {loss:.4f}")

print("predictions:", np.round(y.ravel(), 3))
print("trainable params:", W1.size + b1.size + W2.size + b2.size)
```

使用这个随机种子，训练过程大致是：

```
step 0, loss 1.5909
step 1000, loss 0.0000
step 2000, loss 0.0000
step 3000, loss 0.0000
step 4000, loss 0.0000
step 5000, loss 0.0000
predictions: [0. 1. 1. 0.]
trainable params: 17
```

参数计数：$W_1$ 有 $2\times4=8$ 个，$b_1$ 有 4 个，$W_2$ 有 $4\times1=4$ 个，$b_2$ 有 1 个，共 17 个可训练参数。原稿说“8 参数隐藏层的小网络”，只数了 $W_1$，漏掉了偏置和输出层。

> **这段代码演示了什么、没演示什么**
>
> 它演示了前向、反向、更新和形状对应；没有演示 mini-batch、验证集、学习率调度、正则化、混合精度和分布式。真实训练不是把循环跑 5000 次，而是把数据、模型、优化和系统一起工程化。

## 8.8 算力与缩放

训练一个参数量为 $N$、训练 token 数为 $D$ 的模型，前向计算量约为

$$C_{\text{forward}} \approx 2ND$$

FLOPs。每个参数参与一次乘法和一次加法，通常算 2 FLOPs。反向传播约为前向的 2 倍（计算对激活和参数的梯度），所以

$$C_{\text{train}} \approx 6ND.$$

这是粗略下界，忽略了注意力分数矩阵的 $O(D^2)$ 项、嵌入、归一化、通信和重计算。对长序列，注意力项可能不可忽略。

**缩放定律**研究损失随模型规模、数据量、计算量的变化。Kaplan 等（2020）发现幂律关系；Hoffmann 等（2022，Chinchilla）进一步指出，在固定计算预算下，模型参数和数据量应大致按比例增长，约“每参数 20 个 token”是一个常被引用的经验点。但这不是普遍定律：数据质量、架构、训练目标、推理成本都会改变最优点。

> **“20 tokens/param”不是铁律**
>
> Chinchilla 的结论依赖其数据分布、模型族和计算预算。实际训练还要考虑推理成本、数据重复、领域配比和边际收益。把“20 倍”当成绝对规则，容易做出错误决策。

> **把这一章放回地图**
>
> 训练系统 = 数据 + 模型 + 反向 + 优化 + 系统 + 评估；任何一层都可能成为瓶颈。AdamW、warmup、cosine、梯度裁剪、权重衰减是常见组合，但不是万能配方；混合精度、分布式、调试和评估，是把公式变成可用模型的关键。

# 第 9 章　地图的边缘：生成、自监督与新一代架构

> 这一章不追求细节，只给出“知道有这条路、知道它解决什么问题”的广度。

## 9.1 生成模型：让网络学会造数据

判别模型学习 $p(y \mid x)$，生成模型学习 $p(x)$ 或 $p(x, y)$，并能采样新数据。

**自编码器（AE）** 把输入压缩到低维瓶颈，再重建：

$$z = f_{\text{enc}}(x), \qquad \hat{x} = f_{\text{dec}}(z).$$

损失是重建误差。普通 AE 学到的是压缩表示，但潜空间不一定适合采样；它更像降维/去噪工具。

**变分自编码器（VAE）** 假设潜变量服从先验（通常是高斯），编码器输出后验的均值/方差，解码器从 $z$ 重建数据。损失包括重建项和 KL 正则：

$$\mathcal{L} = \mathbb{E}_{q_\phi(z|x)}\left[-\log p_\theta(x \mid z)\right] + D_{\text{KL}}\left(q_\phi(z \mid x) \,\|\, p(z)\right).$$

VAE 的潜空间连续、可采样，但生成图像往往偏模糊。

**生成对抗网络（GAN）** 同时训练生成器 $G$ 和判别器 $D$：

$$\min_G \max_D \; \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p(z)}[\log(1 - D(G(z)))].$$

GAN 能生成 sharp 的图像，但训练不稳定、容易模式崩溃、评估困难。

**归一化流** 用一系列可逆变换把简单分布映射到复杂分布，可以精确计算似然：

$$\log p(x) = \log p(z) + \log \left| \det \frac{\partial f}{\partial z} \right|.$$

**扩散模型** 定义前向加噪过程，训练网络预测噪声，反向逐步去噪，从高斯噪声生成样本。扩散模型在图像、音频、视频生成上表现很强；代价是采样步数多、计算量大。Latent Diffusion 在潜空间做扩散，降低计算成本。

| 模型 | 核心思想 | 优点 | 局限 |
|---|---|---|---|
| AE | 编码—重建 | 简单、降维 | 不是严格生成模型 |
| VAE | 概率潜变量 + KL | 可采样、连续潜空间 | 生成偏模糊 |
| GAN | 对抗训练 | 样本 sharp | 不稳定、模式崩溃 |
| Flow | 可逆变换 | 精确似然 | 结构受限、计算贵 |
| Diffusion | 加噪—去噪 | 质量高、稳定 | 采样慢、计算大 |

表 9–1　常见生成模型。它们不是互斥的，很多系统会组合使用。

## 9.2 自监督学习：从数据本身找标签

自监督学习用数据的一部分预测另一部分，不需要人工标签。它是现代预训练的核心。

常见代理任务：自回归语言建模（给定前文预测下一个 token，GPT）；掩码语言建模（遮住部分 token 预测内容，BERT）；对比学习（拉近同一图像的不同增强视图，推远不同图像，SimCLR、MoCo）；图像块预测（相对位置、颜色化、拼图）；语音/视频预测未来帧或对比时间片段。

自监督有效，是因为数据本身提供监督信号，规模可以远大于人工标注；学到的表示往往可迁移到下游任务；预训练 + 微调/提示，成为大模型的标准范式。但它不是免费午餐：代理任务与下游任务可能不匹配；数据偏见会被放大；评估需要仔细设计。

## 9.3 图神经网络：处理非网格结构

图像是网格，文本是序列，但很多数据是图：社交网络、分子、知识图谱、推荐系统、代码依赖图。

图神经网络的核心是**消息传递**：

$$\boldsymbol{h}_v^{(l+1)} = \phi\left(\boldsymbol{h}_v^{(l)}, \square_{u\in\mathcal{N}(v)} \psi\left(\boldsymbol{h}_u^{(l)}, \boldsymbol{e}_{u,v}\right)\right),$$

其中 $\mathcal{N}(v)$ 是邻居集合，$\square$ 是聚合（求和、平均、最大），$\psi$ 是消息函数，$\phi$ 是更新函数。

GNN 对节点排列**等变**：换一个节点编号，输出也跟着换，而不是改变结果。这和 CNN 的平移等变、Transformer 的排列等变是同一类思想。代表工作包括 GCN、GAT、GraphSAGE、消息传递网络，应用包括分子性质预测、推荐、欺诈检测、知识图谱推理。

## 9.4 状态空间模型与线性注意力

Transformer 的 $O(n^2)$ 注意力在超长序列上很贵。除了稀疏/线性注意力，另一条路线是**状态空间模型**（SSM）：

$$\boldsymbol{h}_t = \boldsymbol{A}\boldsymbol{h}_{t-1} + \boldsymbol{B}\boldsymbol{x}_t, \qquad \boldsymbol{y}_t = \boldsymbol{C}\boldsymbol{h}_t + \boldsymbol{D}\boldsymbol{x}_t.$$

它和 RNN 一样有递归状态，但可以用卷积/并行扫描高效训练。S4（2022）用结构化状态矩阵和长卷积建模长序列；Mamba（2023）引入**选择性**机制，让状态转移依赖输入，试图兼顾长序列和内容路由。

这类模型的优势是长序列线性/近线性复杂度；挑战是表达力、训练稳定性、硬件友好性和生态。它们不是“Transformer 杀手”，而是长序列工具箱中的另一把工具，实践中也常出现混合架构。

## 9.5 多模态与对齐

多模态模型处理文本、图像、音频、视频、动作等不同模态。常见思路：对比学习（CLIP 用图文对训练图像编码器和文本编码器，让匹配的图文向量接近）；统一 token 化（图像切 patch、音频切帧，投影成 token，与文本一起送入 Transformer）；跨模态注意力（一个模态做 query，另一个做 key/value）；生成式多模态（图像/视频生成、语音合成、视觉问答、具身智能）。

多模态的核心难题不是“把模态拼起来”，而是**对齐**：不同模态的语义粒度、噪声、缺失和时序都不同。评估也比单模态复杂。

预训练语言模型的目标是“预测下一个 token”，不等于“有用、诚实、无害”。对齐技术包括监督微调（SFT）、RLHF（训练奖励模型，再用 PPO 等优化）、DPO（直接用偏好对优化策略）、拒绝采样/最佳-of-n、安全训

练。对齐的评估很难：偏好、正确性、安全性、创造性、多样性往往是多目标；人工评估成本高、偏差大；自动评估容易被“刷分”。

## 9.6 效率与开放问题

效率技术：量化（INT8/INT4/FP8，减少显存和带宽）；剪枝（去掉不重要的权重/头/层）；蒸馏（大模型教小模型）；低秩适配（LoRA，冻结原权重，只训练低秩增量）；MoE（每个 token 只激活部分专家，增加总参数但不线性增加计算）；推理优化（KV Cache、PagedAttention、连续批处理、投机解码、算子融合）。

开放问题包括：推理与规划、幻觉与 grounding、长上下文与记忆、持续学习、可解释性、评估、数据与版权、能效与可持续性、安全与对齐。

## 9.7 如何读一个新架构

遇到一个新模型/论文，按下面的清单问问题：

1. **数据是什么结构？** 网格、序列、图、集合、多模态？
2. **注入了什么归纳偏置？** 局部性、平移等变、顺序、排列等变、稀疏性？
3. **训练目标是什么？** 监督、自监督、生成、对比、强化？
4. **计算/显存复杂度？** 序列长度、batch、参数量如何影响？
5. **工程代价？** 并行、通信、KV Cache、算子支持？
6. **评估是否公平？** 数据、基线、算力、指标是否可比？
7. **失败模式？** 分布外、长尾、对抗、偏见、幻觉？

> **把这一章放回地图**
>
> CNN、RNN、Transformer、SSM、GNN 都是对不同数据结构的回答。选择架构时，先问数据与约束，再问“哪个 SOTA”。架构没有“最新即最好”，只有与数据、算力、延迟和目标的匹配。

# 附录 A　公式与符号速查

## A.1 符号

| 符号 | 含义 |
|---|---|
| $\boldsymbol{x}$ | 输入向量 |
| $\boldsymbol{X}$ | 输入矩阵 / 一个 batch |
| $\boldsymbol{W}, \boldsymbol{b}$ | 权重矩阵、偏置 |
| $\boldsymbol{h}, \boldsymbol{z}$ | 隐藏状态、预激活 |
| $\hat{\boldsymbol{y}}$ | 模型预测 |
| $\mathcal{L}$ | 损失 |
| $\boldsymbol{\theta}$ | 全部可学习参数 |
| $\eta$ | 学习率 |
| $\odot$ | 逐元素乘法 |
| $\top$ | 转置 |
| $\sigma$ | sigmoid 函数 |
| $\mathbf{softmax}$ | 归一化指数函数 |
| $\|\cdot\|_2$ | L2 范数 |
| $\nabla$ | 梯度 |
| $\mathbb{E}$ | 期望 |
| $D_{\mathrm{KL}}$ | KL 散度 |

## A.2 基本公式

线性层

$$\boldsymbol{z} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}, \qquad \boldsymbol{Z} = \boldsymbol{X}\boldsymbol{W} + \boldsymbol{b}.$$

参数量

$$\#\mathbf{params} = n_{\text{in}} n_{\text{out}} + n_{\text{out}}.$$

Sigmoid / Tanh / ReLU

$$\sigma(x) = \frac{1}{1+e^{-x}}, \qquad \sigma'(x) = \sigma(x)(1-\sigma(x)),$$

$$\tanh'(x) = 1 - \tanh^2(x), \qquad \mathrm{ReLU}'(x) = \mathbf{1}[x > 0].$$

Softmax

$$\mathrm{softmax}(\boldsymbol{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

交叉熵

$$\mathcal{L}_{\mathrm{CE}} = -\sum_{k=1}^{K} y_k \log \hat{y}_k.$$

反向传播

$$\boldsymbol{\delta}^{(l)} = \left(\boldsymbol{W}^{(l+1)\top}\boldsymbol{\delta}^{(l+1)}\right) \odot f^{(l)\prime}(\boldsymbol{z}^{(l)}),$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{(l)}} = \boldsymbol{\delta}^{(l)} \boldsymbol{a}^{(l-1)\top}, \qquad \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{(l)}} = \boldsymbol{\delta}^{(l)}.$$

卷积输出尺寸

$$H_{\mathrm{out}} = \left\lfloor \frac{H + 2p - d(k-1) - 1}{s} \right\rfloor + 1.$$

卷积参数量

$$k^2 C_{\mathrm{in}} C_{\mathrm{out}} + C_{\mathrm{out}}.$$

感受野递推

$$\mathrm{RF}_l = \mathrm{RF}_{l-1} + (k_l - 1) d_l J_{l-1}, \qquad J_l = J_{l-1} s_l.$$

简单 RNN

$$\boldsymbol{h}_t = \tanh(\boldsymbol{W}_h \boldsymbol{h}_{t-1} + \boldsymbol{W}_x \boldsymbol{x}_t + \boldsymbol{b}).$$

LSTM

$$\boldsymbol{f}_t = \sigma(\boldsymbol{W}_f [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_f),$$

$$\boldsymbol{i}_t = \sigma(\boldsymbol{W}_i [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_i),$$

$$\tilde{\boldsymbol{c}}_t = \tanh(\boldsymbol{W}_c [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_c),$$

$$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{c}}_t,$$

$$\boldsymbol{o}_t = \sigma(\boldsymbol{W}_o [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_o), \qquad \boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t).$$

Bahdanau 注意力

$$e_{t,j} = \boldsymbol{v}_a^\top \tanh(\boldsymbol{W}_a \boldsymbol{s}_{t-1} + \boldsymbol{U}_a \boldsymbol{h}_j),$$

$$\alpha_{t,j} = \mathrm{softmax}_j(e_{t,j}), \qquad \boldsymbol{c}_t = \sum_j \alpha_{t,j} \boldsymbol{h}_j.$$

缩放点积注意力

$$\mathrm{Attention}(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V})=\mathrm{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\top}}{\sqrt{d_k}}\right)\boldsymbol{V}.$$

Xavier / He

$$\mathrm{Var}(w)=\frac{2}{n_{\text{in}}+n_{\text{out}}}\quad(\text{Xavier}),\qquad \mathrm{Var}(w)=\frac{2}{n_{\text{in}}}\quad(\text{He}).$$

训练算力

$$C_{\text{train}}\approx 6ND.$$

# 附录 B　术语中英对照

| 中文 | 英文 | 一句话解释 |
|---|---|---|
| 感知机 | perceptron | 最早的线性分类器之一 |
| 多层感知机 | multilayer perceptron (MLP) | 多层全连接网络 |
| 激活函数 | activation function | 引入非线性 |
| 前向传播 | forward propagation | 从输入算到输出 |
| 反向传播 | backpropagation | 用链式法则算梯度 |
| 梯度下降 | gradient descent | 沿负梯度更新参数 |
| 学习率 | learning rate | 步长 |
| 过拟合 | overfitting | 记住训练集，泛化差 |
| 正则化 | regularization | 限制模型复杂度 |
| 词嵌入 | word embedding | 词的稠密向量表示 |
| 分布式假设 | distributional hypothesis | 词义由上下文决定 |
| 负采样 | negative sampling | 用二分类近似 softmax |
| 卷积 | convolution / cross-correlation | 局部窗口加权求和 |
| 感受野 | receptive field | 输出点对应的输入区域 |
| 权值共享 | weight sharing | 同一组权重在不同位置复用 |
| 平移等变 | translation equivariance | 输入平移，输出跟着平移 |
| 池化 | pooling | 局部降采样 |
| 循环神经网络 | recurrent neural network (RNN) | 带隐状态的序列模型 |
| 长短期记忆 | long short-term memory (LSTM) | 门控循环网络 |
| 门控循环单元 | gated recurrent unit (GRU) | 更少的门的循环网络 |
| 沿时间反向传播 | backpropagation through time (BPTT) | 沿时间展开的反向传播 |
| 梯度消失 | vanishing gradient | 梯度指数衰减 |
| 梯度爆炸 | exploding gradient | 梯度指数增长 |
| 编码器—解码器 | encoder – decoder | 一个读入，一个生成 |
| 注意力 | attention | 按相关性加权聚合 |
| 自注意力 | self-attention | 序列内部互相注意 |
| 交叉注意力 | cross-attention | 一个序列查询另一个序列 |
| 查询/键/值 | query/key/value | 注意力中的匹配与内容 |
| 多头注意力 | multi-head attention | 多个子空间并行注意 |
| 残差连接 | residual connection | $\boldsymbol{y} = \boldsymbol{F}(\boldsymbol{x}) + \boldsymbol{x}$ |
| 层归一化 | layer normalization | 按特征归一化 |
| 批归一化 | batch normalization | 按 batch 统计归一化 |

| 中文 | 英文 | 一句话解释 |
| --- | --- | --- |
| 混合精度 | mixed precision | FP16/BF16 + FP32 |
| 分布式数据并行 | distributed data parallel | 多卡分 batch，梯度同步 |
| 缩放定律 | scaling law | 损失随规模变化的经验规律 |
| 自监督学习 | self-supervised learning | 从数据构造监督信号 |
| 生成模型 | generative model | 建模数据分布并采样 |
| 图神经网络 | graph neural network | 图上的消息传递 |
| 状态空间模型 | state-space model | 用线性动力系统建模序列 |

# 附录 C　自检题与参考答案

## C.1 第 1 章

1. **神经网络的三个视角是什么？** 函数逼近器、表征学习器、可微计算图。分别回答“能表达什么”“学到了什么”“怎么训练”。

2. **训练与泛化的区别？** 训练是在已见数据上降低损失；泛化是在未见数据上表现好。训练损失低不代表泛化好。

3. **为什么 CNN 不是 Transformer 的必经阶段？** CNN 处理网格数据的视觉支线；Transformer 的主线来自序列建模与注意力。两者是不同归纳偏置，不是线性进化。

## C.2 第 2 章

1. **点积为什么能表示相似度？** $\boldsymbol{x}^\top \boldsymbol{y} = \|\boldsymbol{x}\|\|\boldsymbol{y}\| \cos\theta$，方向越接近，点积越大。

2. **交叉熵与 KL 的关系？** $H(p, q) = H(p) + D_{\mathrm{KL}}(p \| q)$。$H(p)$ 与模型无关，最小化交叉熵等价于最小化 KL。

3. **为什么 softmax 要先减最大值？** 防止 $\exp(z_i)$ 溢出；减最大值不改变结果，且保证至少一项为 1。

## C.3 第 3 章

1. **单层感知机为什么不能表示 XOR？** XOR 四点在平面上对角分布，不存在一条直线把两类分开；可用不等式推出矛盾。

2. **写出一个 XOR 网络。** $h_1 = \mathrm{step}(x_1 + x_2 - 0.5)$（OR），$h_2 = \mathrm{step}(x_1 + x_2 - 1.5)$（AND），$\hat{y} = \mathrm{step}(h_1 - h_2 - 0.5)$。

3. **$784 \to 256 \to 10$ 的参数量？** $784 \times 256 + 256 + 256 \times 10 + 10 = 203{,}530$。

4. **万能逼近定理没有保证什么？** 可学习性、泛化、参数效率（宽度可能很大）。

5. **为什么全零初始化不行？** 同一层神经元输出和梯度完全相同，无法打破对称性。

## C.4 第 4 章

1. **one-hot 的余弦相似度为什么是 0？** 不同词的非零维度不重叠，点积为 0。

2. **负采样如何改写目标？** 把 $V$ 类 softmax 变成“正样本 vs $K$ 个负样本”的二分类，避免对整个词表归一化。

3. **手算更新 $\boldsymbol{v}_c$。** 梯度 $(-0.2689, 0.2311)$，更新后 $(1.0269, -0.0231)$。

4. **嵌入层反向传播为什么稀疏？** 只有被查到的行参与前向，梯度只写到这些行；同一 token 多次出现则累加。权重绑定时输出侧梯度会作用于更多行。

5. **为什么需要位置编码？** 注意力对位置没有内在偏好；没有位置信息，打乱 token 顺序可能不改变结果。

## C.5 第 5 章

1. 全连接层参数量。 $224 \times 224 \times 3 \times 1000 + 1000 = 150{,}529{,}000$。

2. 输出尺寸。 $H = 32, k = 5, s = 2, p = 2$：$(32 + 4 - 5)/2 + 1 = 16$。

3. 卷积是等变还是不变？ 等变：输入平移，输出跟着平移；池化才提供近似不变。

4. 两个 $3 \times 3$ vs 一个 $5 \times 5$。 感受野都为 $5 \times 5$；参数量 18 vs 25（同通道、忽略偏置）；两个 $3 \times 3$ 多一次非线性。

5. AlexNet 的 26%→16%。 Top-5 错误率；26.2% 是当年第二名，15.3% 是 7 模型集成，16.4% 是 5 模型平均。不能与 top-1 混用。

## C.6 第 6 章

1. RNN 手算。 $h_1 = 0.9640$，$h_2 = 0.9018$，$h_3 = 0.8959$。

2. 为什么 tanh 有界但梯度会消失？ 前向状态有界；反向乘 $W_h \tanh'$，$\tanh'$ 可接近 0，$W_h$ 可小于 1，连乘指数衰减。

3. LSTM 三个门。 遗忘门控制旧细胞状态保留，输入门控制新信息写入，输出门控制当前隐藏状态暴露。

4. LSTM 为什么有帮助？ 细胞状态加性更新，$\partial c_t / \partial c_{t-1} = f_t$；$f_t \approx 1$ 时梯度路径接近恒等。但不是永远恒等。

5. KV Cache 与 RNN 状态的区别。 RNN 压缩历史为固定状态；KV Cache 显式保存所有历史 Key/Value，内存随上下文增长。

## C.7 第 7 章

1. Bahdanau 三步。 打分 $e_{t,j} = v_a^\top \tanh(W_a s_{t-1} + U_a h_j)$；softmax 得 $\alpha$；$c_t = \sum_j \alpha_{t,j} h_j$。

2. 手算上下文。 $\alpha = (0.212, 0.212, 0.576)$，$c = (1.000, 0.576)$。

3. 为什么除以 $\sqrt{d_k}$？ 点积方差随 $d_k$ 增长，softmax 容易饱和；缩放让方差回到 1 附近。

4. 三种注意力。 自注意力：同一序列；交叉注意力：解码器查询编码器；掩码注意力：限制可见位置。

5. Transformer 的优势与代价。 优势：并行、短路径、灵活；代价：$O(n^2)$、数据需求、位置编码、KV Cache。

## C.8 第 8 章

1. 训练系统六层。 数据、模型、反向、优化、系统、评估。

2. AdamW 与 L2 的区别。 AdamW 把权重衰减从梯度中解耦，直接作用于参数；L2 把惩罚加进损失，和自适应学习率耦合。

3. BatchNorm vs LayerNorm。 BatchNorm 统计 batch 维度（每通道）；LayerNorm 统计每个样本的特征维度，不依赖 batch。

4. FP16 vs BF16。 FP16 精度高、范围小，需 loss scaling；BF16 范围大、精度低，通常无需 scaling。

5. $6ND$ 的来源。 前向约 $2ND$，反向约 $4ND$，合计 $6ND$；忽略注意力二次项、通信、重计算等。

# 附录 D　延伸阅读

## D.1 教材与课程

- Goodfellow, Bengio, Courville. *Deep Learning*. MIT Press, 2016.
- Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- Prince. *Understanding Deep Learning*. MIT Press, 2023.
- Sutton, Barto. *Reinforcement Learning: An Introduction*. 2nd ed., MIT Press, 2018.
- *Dive into Deep Learning*（d2l.ai）。
- Stanford CS231n（视觉）、CS224n（NLP）、CS336（语言模型从零实现）。

## D.2 关键论文

### 基础与历史

- McCulloch, Pitts. “A Logical Calculus of the Ideas Immanent in Nervous Activity.” 1943.
- Rosenblatt. “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain.” 1958.
- Minsky, Papert. *Perceptrons*. 1969.
- Rumelhart, Hinton, Williams. “Learning Representations by Back-Propagating Errors.” *Nature*, 1986.
- Cybenko. “Approximation by Superpositions of a Sigmoidal Function.” 1989.
- Hornik, Stinchcombe, White. “Multilayer Feedforward Networks are Universal Approximators.” 1989.

### 视觉与序列

- LeCun et al. “Gradient-Based Learning Applied to Document Recognition.” 1998.
- Krizhevsky, Sutskever, Hinton. “ImageNet Classification with Deep Convolutional Neural Networks.” 2012.
- He et al. “Deep Residual Learning for Image Recognition.” 2016.
- Hochreiter, Schmidhuber. “Long Short-Term Memory.” 1997.
- Gers, Schmidhuber, Cummins. “Learning to Forget: Continual Prediction with LSTM.” 2000.
- Cho et al. “Learning Phrase Representations using RNN Encoder – Decoder for Statistical Machine Translation.” 2014.
- Sutskever, Vinyals, Le. “Sequence to Sequence Learning with Neural Networks.” 2014.
- Bahdanau, Cho, Bengio. “Neural Machine Translation by Jointly Learning to Align and Translate.” 2015.
- Luong, Pham, Manning. “Effective Approaches to Attention-based Neural Machine Translation.” 2015.

### 表示与 Transformer

- Mikolov et al. “Efficient Estimation of Word Representations in Vector Space.” 2013.
- Pennington, Socher, Manning. “GloVe: Global Vectors for Word Representation.” 2014.
- Vaswani et al. “Attention Is All You Need.” 2017.

- Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” 2019.
- Radford et al. “Language Models are Unsupervised Multitask Learners.” 2019.
- Brown et al. “Language Models are Few-Shot Learners.” 2020.
- Dosovitskiy et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.” 2021.

**规模、生成与对齐**

- Kaplan et al. “Scaling Laws for Neural Language Models.” 2020.
- Hoffmann et al. “Training Compute-Optimal Large Language Models.” 2022.
- Ho, Jain, Abbeel. “Denoising Diffusion Probabilistic Models.” 2020.
- Rombach et al. “High-Resolution Image Synthesis with Latent Diffusion Models.” 2022.
- Ouyang et al. “Training Language Models to Follow Instructions with Human Feedback.” 2022.
- Rafailov et al. “Direct Preference Optimization.” 2023.
- Dao et al. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.” 2022.
- Gu, Dao. “Mamba: Linear-Time Sequence Modeling with Selective State Spaces.” 2023.

## D.3 怎么读论文

1. 先读摘要、图和结论，回答“它解决什么问题”；
2. 看方法图的输入/输出/损失，不要一上来抠公式；
3. 找到它与已有方法的差异（归纳偏置、目标、计算）；
4. 看实验：数据、基线、算力、指标是否公平；
5. 看局限：作者自己承认什么，没承认什么；
6. 用自己的话写三句话总结，再决定是否复现。

> **关于原稿的错误**
>
> 本册正文已修正原稿中的主要事实与计算错误。完整的逐项对照、页码和修订说明，见独立的《审校报告》，不塞进正文，避免打断阅读。